

HW #1

09/06/2026 23:41:30

<https://chatgpt.com/g/g-p-6a98ccc640548191a0af17704cc0651d/c/6a9dfe88-be78-83e8-98bc-e51eee03fb20>

그냥 처음부터 다 다시할까?

응. 이번에는 처음부터 다시 하는 쪽이 오히려 더 빠를 것 같아. 다만 진짜로 모든 걸 버리...

좋아 첨부부터 다시 고고해보자

좋아. 이번엔 예전 결과 숫자는 전부 잊고, 교수님 요구사항만 들고 완전히 새 프로젝트처...

오케이. README랑 DESIGN은 이상한게 없는 이상 그대로 쓰면 될것 같아. 우리 같이 ...

좋아. 이번엔 기존 README와 DESIGN은 문제 없으면 유지하고, 우리가 지금부터 쓸 새 ...

Thinking ● 시작 방식 정리하기 GitHub 저장소에 직접 파일을 쓰는 연결은 현재 사용할...

Assignment 1 로 만들었어 ㅎㅎ

ㅋㅋ 좋아. 그럼 폴더 이름은 Assignment 1 그대로 가자. 전혀 문제 없어. 다만 터미널에...

잠깐! 일단 바탕화면에 CMPE-255 폴더를 만들었는데 여기에다가 touch 명령어를 하면...

응 ㅎㅎ 그 CMPE-255 폴더를 네 로컬 작업 폴더로 쓸 거면 거기서 하면 돼. 먼저 터미널...

깃헙에 연결해야해. 그리고 하위 폴더면 어디를 말하는거야?

응, 먼저 GitHub repo랑 로컬 폴더를 연결하는 게 먼저야. 그리고 내가 말한 “하위 폴더”...

changhyunkim\@Mac CMPE-255 % git clone...

응, 에러 원인은 간단해 ㅎㅎ 마지막에 .을 붙였기 때문에 Git이 현재 CMPE-255 폴더 안...

changhyunkim\@Mac CMPE-255 % ls -la total 16 drwxr-xr-x 5...

Thinking ● 시작 위치 확인하기 현재 폴더에 기존 과제가 있으므로, 먼저 원격 저장소를 ...

아 잠깐만 폴더 이름이 헛갈렸다 CMPE-272로 해야해

아 ㅋㅋ 잠깐, 여기서는 그렇게 바꾸면 오히려 더 헛갈려. 지금 우리가 다시 하는 과제는...

changhyunkim\@Mac CMPE-255 % ls -la total 16 drwxr-xr-x 4...

좋아. 이제 깔끔해졌어 ㅎㅎ 현재 CMPE-255 안에는: CMPE-255/ └─ Assignment 1/...

Clone 했어. 근데 아까 만들어둔 폴더들 in Assignemtn 1 다시 깃헙 클론한 데로 옮겨...

Thinking ● 정리하며 이동하기 먼저 저장소 안에 같은 이름의 폴더가 있는지 확인하고 있...

changhyunkim\@Mac CMPE-255 % ls -la total 16 drwxr-xr-x 4...

오케이, 지금 출력 보니까 바깥쪽에 있던 Assignment 1 폴더는 현재 안 보여. 지금 CMP...
응 내가 지워버리고 클론한 폴더에 있는 Assignment 1으로 다 옮겼어
좋아, 그럼 이제 제일 깔끔한 상태야 ㅎㅎ 앞으로는 clone된 repo 안의 Assignment 1만...
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining % ls -la total 8 drwx...
응, 그게 제일 안전해. 🍌 지금 구조는: FA2026_CMPE-255-Data-Mining/ |—...
changhyunkim\@Mac Assignment 1 % find . -maxdepth 4 -print | sort
오케이, 지금 Assignment 1은 거의 빈 폴더네 ㅎㅎ 그리고 파일명이 README.md로 ...
changhyunkim\@Mac Assignment 1 % find . -maxdepth 4 -print | sort
완벽해 ㅎㅎ Part 1 골격은 제대로 만들어졌어. 이제 이 Assignment 1 폴더만 앞으로 ...
좋아 스펙.엠디로 ㄱㄱ
좋아, 바로 spec.md 가자. 이 파일은 ****“무엇을 어떻게 분석할 것인가”**를 정하는 문서...
플랜 ㄱㄱ
좋아, plan.md 가자. 이걸 이제 실제로 우리가 어떤 순서로 작업할지 체크리스트로 쪼개...
ㄱㄱ
좋아. 이제부터는 문서 설계 끝 → 실제 Part 1 분석 시작이야. 이번엔 한 단계씩 실행하고 ...
import pandas as pd import numpy as np import matplotlib.pyplot as plt ...
응, 당연히 가능해. 지금 .ipynb 파일로 작업 중이면 VS Code에서도 Colab처럼 셀 단위...
****Running cells with 'Python 3.13.0' requires the ipykernel package.** 라고 뜨...
응, 이걸 흔한 에러야. 지금 VS Code가 선택한 Python 환경에 ipykernel이 없어서...
Text 파일은 이제 실행되는데 파이썬 코드들이 안된다
아 그럼 지금 Markdown/Text 셀은 보이는데 Python kernel이 아직 제대로 연결되지 ...
print("Hello CMPE 255!") 실행되는데 --- ModuleNotFoundError Traceback (mo...
좋아, 이제 원인이 확실해졌어. print("Hello CMPE 255!")는 실행되니까 kernel 자체...
아 다른경로가 나오네
오케이, 그럼 딱 원인 잡혔어 ㅎㅎ VS Code notebook이 네 .venv가 아니라 다른 Pyth...
/Users/changhyunkim/Documents/CMPE-272-Enterprise-Software-...
응, 원인 100% 확실해졌어 ㅋㅋ 지금 CMPE-255 notebook이 ****예전 CMPE-272...**
URL을 입력하라는데!/?
아 ㅋㅋ 그럼 지금 “Jupyter Server URL” 쪽으로 들어간 거야. 그건 우리가 원하는 메뉴...

뭐지 계속 272 여기에 남아있어

Thinking ● 커널 고정 문제 해결하기 VS Code가 이전 프로젝트의 커널을 계속 선택하...

(.venv) changhyunkim\@Mac house-prices % jupyter kernelspec list...

좋아, 이걸 255 커널이 제대로 등록된 상태야. 🍌 지금 중요한 건 두 개 다 255 쪽이라는 ...

/Users/changhyunkim/Desktop/CMPE-255/FA2026_CMPE-255-Data-...

응, 완전히 제대로 연결됐어. 🍌 지금 notebook이 정확히 이 환경을 쓰고 있어: .../CMP...

오케이 된다! 근데 그냥 구글 Colab에서 하는게 편했나ㅋㅋ 근데 ...

ㅋㅋ Colab이 확실히 초반 세팅은 더 편해.근데 지금처럼 GitHub 프로젝트 구조랑 .ven...

좋아 다 실행잘됐어

좋아, 이제 Data Understanding의 첫 체크는 완료야. 다음은 결과를 그냥 보고 넘어가...

Dataset shape: (1460, 81) Data type counts: str 43 int64 35 float64 3...

Thinking ● 요약 해석하기 출력상 1,460개 주택과 80개 설명변수, 목표변수...

np.float64(1.8828757597682129)

좋아. skewness ≈ 1.88 이면 **SalePrice가 꽤 강하게 오른쪽으로 치우쳐 있다(right-...

ㄱㄱ

좋아, 다음은 Missing Values Analysis 가자. 이번 목표는 단순히 "NaN이 많다"가 아...

[Image: 스크린샷 2026-09-06 오후 10.03.39.png]

Thinking ● 해석 정리하기 현재 세 열의 결측값은 실제 누락보다 구조적 결측으로 보이고...

[Image: 스크린샷 2026-09-06 오후 10.04.46.png]

Thinking ● 구분 정리하기 `keep\_default\_na=False`를 사용하면 실제 결측치와...

KeyError: 'PandasMissing'

응, 이걸 네 데이터 문제가 아니라 내 코드에서 문자열 컬럼을 dtype == "object"로만 ...

[Image: 스크린샷 2026-09-06 오후 10.06.33.png]

Thinking ● 정제 정책 설정하기 구조적 부재와 실제 결측을 구분하는 전처리 방침을 세우...

[Image: 스크린샷 2026-09-06 오후 10.09.33.png] 와 근데 따라는 가고 있는데, 뭐...

ㅋㅋ 지금 딱 멈추는 게 맞아. 코드가 너무 빨리 복잡해졌어. 지금까지 한 걸 머릿속 그림으...

허허 이해는 나중에 하고 일단 숙제 완료부터하자

좋아 ㅋㅋ 그럼 이해는 나중에 복습용으로 따로 정리하고, 지금은 제출 가능한 완성본 만드...

SalePrice 1.000000 OverallQual 0.790982 GrLivArea 0.708624 GarageCar...

좋아. 결과도 정상적이야. 핵심은 OverallQual (0.79)와 GrLivArea (0.71)가 가격과 ...

[Image: 스크린샷 2026-09-06 오후 10.25.51.png]

좋아. 이걸 결과가 아주 깔끔해. Gradient Boosting이 현재 validation 기준 1등이야. ...

Fold RMSE: [23534.66421654 32806.16651018 27905.73560421...

좋아. 이 정도면 Gradient Boosting이 우연히 한 split에서만 잘 나온 건 아니라는 근거...

[Image: 스크린샷 2026-09-06 오후 10.29.48.png]

좋아, 이 결과도 충분히 쓸 만해. 여기서 보이는 핵심은 큰 오차가 주로 비싼 집들에서 발생...

[Image: 스크린샷 2026-09-06 오후 10.31.15.png]

좋아. 이걸로 Part 1 분석의 핵심 결과는 사실상 다 나왔어. 이제 정리하고 저장만 하면 돼....

``` figures/ ``` 여기에 지금 feature\\_importance.png 밖에 없어..

응 ㅋㅋ 정상(?)이야. 우리가 앞에서 그래프들은 화면에만 plt.show() 했고 파일로 저장은...

기존 셀들이 어떤것들일까?

응. 우리가 앞에서 이미 만든 그래프 셀은 딱 이 4개였어. VS Code notebook에서 아래 ...

좋아 끝

좋아 ㅋㅋ 그럼 Part 1 분석은 거의 끝났어. 이제 남은 건 정리와 제출 준비야. 순서는:...

(.venv) changhyunkim\@Mac house-prices % ls -lh results total 80 -rw-r--r...

좋아. 파일 저장 상태는 정상이고 Part 1 분석 artifact도 충분히 갖춰졌어. 🍊 현재:...

results\\_df.to\\_csv( &#x20; "../results/model\\_comparison.csv", &#x20;...

Thinking ● 정리 방향 확인하기 저장 코드 셀은 삭제하지 않는 편이 좋아요. 실행 결과를...

올렸어

Thinking 🌐 웹 검색 중 ● 확인 방법 정리 제공된 저장소 경로를 직접 확인하려 했지만 ...

changhyunkim\@Mac FA2026\\_CMPE-255-Data-Mining % python - <<'PY'...

응, 이걸 간단해 ㅎㅎ 지금 repo root에서는 .venv가 활성화되어 있지 않아서 python 명...

출력 없음 까지 확인했어

Thinking 🌐 웹 검색 중 ● 웹 검색 중 ✓ 몇 초간 처리함 좋아. 그럼 최소한 .venv는...

changhyunkim\@Mac FA2026\\_CMPE-255-Data-Mining % git status ...

Thinking 🌐 웹 검색 중 ● 웹 검색 중 ✓ 8s 동안 처리함 좋아, 이제 거의 다 확인됐어. ...

changhyunkim\@Mac FA2026\\_CMPE-255-Data-Mining % python3 - <<'PY'...

좋아, 거의 완벽해. 53개 코드 셀 중 52개 실행, output 있는 셀은 36개면 정상 범위야. ...





그냥 처음부터 다 다시할까?

2026. 9. 6. 오후 9:00:55



응. 이번에는 **처음부터 다시 하는 쪽이 오히려 더 빠를 것 같아**. 다만 진짜로 모든 걸 버리는 건 아니고, **\*\*“결과물은 새로 만들고, 요구사항/구조/데이터셋 선택은 재사용”\*\***하는 방식이 제일 좋아.

지금 기존 작업은 여기저기 숫자가 다르고, Part 2 artifact도 완전히 복구된 상태가 아니고, notebook 실행 흔적도 불완전해서 계속 수선하다 보면 오히려 더 헷갈릴 가능성이 커. 특히 너도 이 과제를 나중에 직접 설명할 수 있어야 하니까, 이번에 처음부터 한 번 제대로 따라가면서 만드는 게 의미가 있어.

내 추천은 이렇게야:

1. 기존 **Assignment1** 과 현재 **Assignment-1** 은 백업으로 그대로 둔다.
2. 새 **Assignment-1** 안에서 실제 제출용 결과는 처음부터 다시 만든다.
3. Part 1은 **House Prices**를 그대로 사용하되, 새 notebook에서 EDA → preprocessing → modeling → evaluation → interpretation을 처음부터 다시 실행한다.
4. Part 2도 **Customer Segmentation / Market Basket / Anomaly Detection** 세 개를 각각 처음부터 다시 실행한다.
5. 이번에는 모든 프로젝트에 같은 형식을 적용한다: **intent → spec → plan → notebook/code → results → README**.
6. 숫자, 그래프, README 설명은 반드시 **마지막 실행 결과 하나만 기준으로** 한다.
7. 마지막에 Part 1 / Part 2 YouTube를 찍고 링크를 README에 넣는다.

그리고 중요한 건 **예전 결과를 참고는 하되 복붙하지 않는 것**이야. 예를 들어 예전에 Customer Segmentation에서 **k=6**이 나왔더라도, 이번에는 먼저 **k=2~10**을 다시 돌리고 silhouette/elbow를 보고 **다시 k를 결정**하는 식으로 가자. 결과가 같으면 좋은 거고, 달라도 문제 없어. 오히려 그게 reproducibility 확인이야.

Part 1도 마찬가지야. 예전 RMSE 숫자는 전부 잊고, 새 notebook을 실행해서 나온 값만 최종값으로 쓰자. 이번에 아까 본 Antigravity 흐름을 그대로 넣으면 더 좋아.

Professor Requirements

↓

intent.md

↓

spec.md

↓

plan.md

↓

Notebook / Code

↓

Validation

↓

Results

↓

README

↓

YouTube

이렇게 하면 단순히 숙제를 다시 하는 게 아니라 **AI-assisted data science** 프로젝트를 제대로 한 번 수행하는 것이 돼.

그리고 나는 **Part 1**부터 다시 시작하는 것을 추천해. Part 1 하나를 완성형 template으로 만든 다음 Part 2 세 개에 그대로 적용하면 속도가 훨씬 빨라져.

첫 단계에서는 코드조차 아직 안 짤 거야.

**House Prices Part 1**의 `intent.md` → `spec.md` → `plan.md` 를 먼저 확정하고, 그 다음 새 **notebook**을 처음부터 한 셀씩 실행하자.

이번에는 내가 각 단계에서 “지금 왜 이걸 하는지”까지 같이 설명하면서 가자. 그러면 YouTube 찍을 때도 거의 그대로 네 설명이 될 거야.

2026. 9. 6. 오후 9:01:05



좋아 첨부터 다시 고고해보자

2026. 9. 6. 오후 9:01:46



좋아. 이번엔 **예전 결과 숫자는 전부 잊고**, 교수님 요구사항만 들고 완전히 새 프로젝트처럼 가자. 😊  
단, dataset 선택과 좋은 아이디어만 재사용하고 **분석 결과는 새 실행에서 나온 것만 최종 제출물로 인정하**  
**는** 걸로.

우선 전체 순서를 이렇게 고정하자.

#### Assignment 1

- |
- |— Part 1 — House Prices
  - | 1. Intent
  - | 2. Specification
  - | 3. Plan
  - | 4. Data Understanding / EDA
  - | 5. Data Preparation
  - | 6. Modeling
  - | 7. Evaluation
  - | 8. Interpretation
  - | 9. Final Notebook + Results
  - | 10. README
  - | 11. Chat Transcript
  - | 12. YouTube
  - |
- |— Part 2 — Professor Experiment Replication
  - |— Customer Segmentation
  - |— Market Basket Analysis
  - |— Anomaly Detection

그리고 지금은 **Part 1**만 본다.

## Step 1 — Part 1 Intent



우리가 처음 만들 파일은:

```
Assignment-1/
└─ Part-1/
 └─ house-prices/
 └─ intent.md
```

내용은 이렇게 가면 좋아.

```
Part 1 – Project Intent
```

```
Project Title
```

```
AI-Assisted House Price Prediction
```

```
Objective
```

The goal of this project is to build an end-to-end data science workflow that predicts residential house prices using the Kaggle House Prices dataset.

The project will use an AI coding assistant throughout the data science lifecycle while maintaining human understanding and validation of the generated analysis.

```
Business Problem
```

House prices depend on many factors such as location, overall quality, living area, age, garage capacity, and other property characteristics.

The objective is to determine whether these characteristics can be used to predict a home's sale price accurately.

```
Dataset
```

```
Kaggle House Prices:
```

```
Advanced Regression Techniques
```

Target variable:

- SalePrice

The dataset includes numerical and categorical features describing residential properties.

## ## Main Question

Can we build a machine learning model that predicts SalePrice from the available property characteristics with reasonable accuracy?

## ## Data Science Approach

The project will follow a structured data science workflow inspired by CRISP-DM:

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Interpretation and Communication

## ## Role of AI

ChatGPT / an agentic coding assistant will help with:

- planning the analysis
- generating and reviewing Python code
- suggesting preprocessing approaches
- suggesting machine learning models
- debugging
- interpreting model results
- organizing project documentation

AI-generated outputs will be reviewed and explained in my own

words rather than accepted without validation.

## ## Success Criteria

The project will be considered successful if:

- the dataset is explored and understood
- missing and categorical data are handled appropriately
- multiple regression approaches are evaluated
- model performance is measured using appropriate metrics
- the final model is compared against a baseline
- important features and prediction errors are interpreted
- the entire workflow is reproducible
- the final results can be clearly explained without relying on the AI-generated text

여기서 중요한 게 마지막 두 문장이야.

교수님이 특별히:

***read and understand and provide a paraphrase of the output***

이라고 했잖아.

그래서 우리는 이번 프로젝트의 원칙 자체에

***AI output을 그대로 제출하지 않고 내가 검증하고 내 말로 설명한다***

를 박아놓는 거야.

## Step 2 — Specification

그 다음:

spec.md

여기에는 **\*\*“그래서 구체적으로 어떤 분석을 할 건데?”\*\***를 적는다.

아직 특정 모델이 반드시 우세하다고 정하지 않는다.

## # Part 1 – Analysis Specification

### ## 1. Problem Type

Supervised Machine Learning

Task:

Regression

Target:

SalePrice

---

### ## 2. Dataset Validation

Before modeling:

- inspect dataset shape
- inspect column types
- check missing values
- check duplicates
- inspect target distribution
- examine numerical and categorical features

---

### ## 3. Exploratory Data Analysis

EDA will include:

- SalePrice distribution
- summary statistics
- missing-value analysis
- correlation analysis
- relationships between important features and SalePrice

- potential outliers

Example features to investigate may include:

- OverallQual
- GrLivArea
- GarageCars
- TotalBsmtSF
- YearBuilt

These features are examples only and will be validated using the actual dataset.

---

## ## 4. Data Preparation

The preprocessing pipeline should support both:

### ### Numerical Features

- missing-value imputation
- optional transformations when justified
- scaling when required by the model

### ### Categorical Features

- missing-value handling
- categorical encoding

Preprocessing should be performed using reusable pipelines where practical.

---

## ## 5. Train / Validation Strategy

The dataset will be divided into training and validation data.

Random state will be fixed for reproducibility.

Cross-validation may also be used for more reliable model comparison.

---

## ## 6. Models

We will begin with a simple baseline and then compare stronger regression models.

Candidate models:

1. Dummy / simple baseline
2. Linear Regression or regularized linear regression
3. Random Forest Regressor
4. Gradient Boosting model

Additional models will only be introduced when they provide a clear analytical benefit.

---

## ## 7. Evaluation Metrics

Main metrics:

- RMSE
- MAE

Optional supporting metric:

- $R^2$

The final model will not be selected from a single metric without considering model behavior and validation results.

---

## ## 8. Model Interpretation

The analysis should include:

- comparison of model performance
- important predictive features
- inspection of large prediction errors
- limitations of the model

---

## ## 9. Required Final Artifacts

The project should produce:

- executed Jupyter notebook
- Python environment / requirements
- dataset or dataset instructions
- figures
- model comparison results
- written interpretation
- README
- AI conversation transcript
- YouTube walkthrough link

이걸 보면 아까 네가 보여준 그림의 `spec.md` 역할이 딱 보이지?

`intent.md`

= 왜 하는가

`spec.md`

= 무엇을 해야 하는가

---

## Step 3 — Plan

그리고 실제 AI agent가 작업하기 좋은 크기로 나눈다.

## # Part 1 – Implementation Plan

### ## Phase 1 – Dataset Understanding

- [ ] Load train.csv
- [ ] Inspect shape and columns
- [ ] Inspect data types
- [ ] Check duplicates
- [ ] Check missing values
- [ ] Examine SalePrice

### ## Phase 2 – Exploratory Data Analysis

- [ ] Plot SalePrice distribution
- [ ] Analyze missing values
- [ ] Analyze numerical correlations
- [ ] Examine important relationships
- [ ] Investigate potential outliers
- [ ] Summarize findings

### ## Phase 3 – Data Preparation

- [ ] Separate X and y
- [ ] Split train and validation data
- [ ] Build numerical preprocessing
- [ ] Build categorical preprocessing
- [ ] Combine preprocessing using ColumnTransformer
- [ ] Validate transformed data

### ## Phase 4 – Baseline Modeling

- [ ] Establish baseline
- [ ] Train first regression model
- [ ] Calculate RMSE / MAE
- [ ] Verify predictions

### ## Phase 5 – Model Comparison



- [ ] Train additional regression models
- [ ] Compare validation metrics
- [ ] Perform cross-validation where useful
- [ ] Select final model

## ## Phase 6 – Interpretation

- [ ] Identify important features
- [ ] Compare predicted vs actual prices
- [ ] Examine large prediction errors
- [ ] Discuss limitations

## ## Phase 7 – Finalization

- [ ] Restart notebook kernel
- [ ] Run All
- [ ] Verify no errors
- [ ] Save notebook outputs
- [ ] Export figures/results
- [ ] Update README
- [ ] Add chat transcript
- [ ] Record YouTube walkthrough
- [ ] Add YouTube URL to README

---

## 그리고 이번에 중요한 규칙 하나

아직 `kaggle notebook` 수준으로 50개 차트를 만들지 말자.

교수님 과제니까 우리가 설명할 수 있는 것만 한다.

예를 들어 EDA에서:

SalePrice Distribution

Missing Values

Correlation

OverallQual vs SalePrice

GrLivArea vs SalePrice

정도부터 시작하고,

그걸 보고:

“왜 다음 분석이 필요한지”

판단한다.

즉,

❌ AI가 300줄짜리 notebook 생성 → 우리는 읽어보기  
가 아니라

✅ 한 단계 실행 → 결과 확인 → 이해 → 다음 단계  
방식으로 가자.

## 이제 실제 분석 시작

다음 단계는 드디어 Python이다.

### Phase 1

```
import pandas as pd

df = pd.read_csv("train.csv")

df.head()
```

그리고 처음 확인할 건 딱 이것뿐이야.

```
df.shape
df.info()
df.isnull().sum().sort_values(ascending=False).head(20)
df.duplicated().sum()
df["SalePrice"].describe()
```

아직 머신러닝 절대 안 한다.

먼저 이 결과를 보고 우리가:

“이 데이터가 도대체 어떻게 생겨먹었는가?”

부터 이해하자.

이번에는 네가 코드를 따라가면서 왜 하는지 모르는 줄이 없도록 진행하자.

다음은 `train.csv` 를 가지고 **Phase 1 — Data Understanding**부터 실제로 시작하면 된다.

여기부터는 Colab이나 VS Code에서 하나씩 같이 실행해보자.

2026. 9. 6. 오후 9:01:53



오케이. README랑 DESIGN은 이상한게 없는 이상 그대로 쓰면 될것 같아. 우리 같이 깃헙에 폴더들 파일들 올려놓고 시작하자

2026. 9. 6. 오후 9:10:14



좋아. 이번엔 기존 **README** 와 **DESIGN** 은 문제 없으면 유지하고, 우리가 지금부터 쓸 새 폴더/파일만 GitHub에 깔아두고 시작하면 돼. GitHub를 직접 수정하려면 연결된 GitHub 접근이 있으면 가장 깔끔해서, 그걸 먼저 붙이는 게 좋아.

2026. 9. 6. 오후 9:10:21



## Thinking

- 시작 방식 정리하기

GitHub 저장소에 직접 파일을 쓰는 연결은 현재 사용할 수 없어서, 내가 폴더 구조와 파일 내용을 단계별로 안내하고 사용자가 터미널에서 실행하는 방식으로 진행하겠습니다.

- 시작 방식 정리 완료

✓ 11s 동안 처리함

좋아. **README**와 **DESIGN**은 건드리지 않고, 제출용 골격만 먼저 만들자.

지금 GitHub 연결은 했지만 이 채팅에서는 GitHub 쓰기 도구가 실제로 노출되지 않아서 내가 직접 commit/push는 못 해. 대신 네 Mac 터미널에서 딱 이 명령들만 실행하면 안전하게 구조가 만들어져. 기존 파일은 덮어쓰지 않아.

먼저 repo 루트로 가서:

```
cd \~/path/to/FA2026_CMPE-255-Data-Mining
git pull
cd Assignment-1
```

그 다음 폴더 생성:

```
mkdir -p Part-1/house-prices/notebooks
mkdir -p Part-1/house-prices/data
mkdir -p Part-1/house-prices/figures
mkdir -p Part-1/house-prices/results
mkdir -p Part-1/house-prices/chat-transcript
mkdir -p Part-1/house-prices/report

mkdir -p Part-2/customer-segmentation
mkdir -p Part-2/market-basket-analysis
mkdir -p Part-2/anomaly-detection
```

Git은 빈 폴더를 저장하지 않기 때문에 `.gitkeep` 도 넣자:

```
touch Part-1/house-prices/notebooks/.gitkeep
touch Part-1/house-prices/data/.gitkeep
touch Part-1/house-prices/figures/.gitkeep
touch Part-1/house-prices/results/.gitkeep
touch Part-1/house-prices/chat-transcript/.gitkeep
touch Part-1/house-prices/report/.gitkeep
```

그리고 우리가 아까 만든 세 문서는 기존 파일이 없을 때만 생성:

```
[-e Part-1/house-prices/intent.md] || touch Part-1/house-prices/intent.md
[-e Part-1/house-prices/spec.md] || touch Part-1/house-prices/spec.md
[-e Part-1/house-prices/plan.md] || touch Part-1/house-prices/plan.md
```

그러면 일단 구조는:

```
Assignment-1/
|
├─ README.md ← 유지
├─ DESIGN... ← 유지
|
├─ Part-1/
| └─ house-prices/
| ├── intent.md
| └─ spec.md
```

```
| |— plan.md
| |— notebooks/
| |— data/
| |— figures/
| |— results/
| |— chat-transcript/
| |— report/
|
|— Part-2/
 |— customer-segmentation/
 |— market-basket-analysis/
 |— anomaly-detection/
```

여기까지 만든 뒤 먼저 확인:

```
find . -maxdepth 4 -type f | sort
```

그리고:

```
git status
```

아직 **commit**하지 말고, 저 두 명령 결과를 여기 붙여줘.

그걸 내가 한번 보고 이상 없으면 바로 다음으로 `intent.md` → `spec.md` → `plan.md` 내용 넣고 첫 **commit** 하자. 그 다음 진짜 `train.csv` 부터 시작하면 돼.

2026. 9. 6. 오후 9:10:38



Assignment 1 로 만들었어 ㅎㅎ

2026. 9. 6. 오후 9:11:30



ㅋㅋ 좋아. 그럼 폴더 이름은 `Assignment 1` 그대로 가자. 전혀 문제 없어.

다만 터미널에서는 공백 때문에 항상 따옴표를 쓰거나 폴더 안으로 먼저 들어가면 돼.

```
cd "Assignment 1"
```

이제 여기 안에서 Part 1 골격부터 만들자.

```
mkdir -p Part-1/house-prices/{notebooks,data,figures,results,chat-transcript,report}
```

그리고 문서 3개:

```
touch Part-1/house-prices/intent.md
touch Part-1/house-prices/spec.md
touch Part-1/house-prices/plan.md
```

빈 폴더도 GitHub에 올라가게:

```
touch Part-1/house-prices/notebooks/.gitkeep
touch Part-1/house-prices/data/.gitkeep
touch Part-1/house-prices/figures/.gitkeep
touch Part-1/house-prices/results/.gitkeep
touch Part-1/house-prices/chat-transcript/.gitkeep
touch Part-1/house-prices/report/.gitkeep
```

그러면 우선:

```
Assignment 1/
├─ README.md
├─ DESIGN...
```



```
|
└─ Part-1/
 └─ house-prices/
 ├── intent.md
 ├── spec.md
 ├── plan.md
 ├── notebooks/
 ├── data/
 ├── figures/
 ├── results/
 ├── chat-transcript/
 └─ report/
```

가 되는 거야.

그리고 **아직 Part 2는 만들지 말자**. 이번엔 Part 1을 완성형 template으로 만든 뒤 그대로 복제하는 편이 덜 헛갈릴 것 같아.

폴더 만들었으면 다음엔 내가 아까 작성한 `intent.md` 부터 실제 파일에 넣을 내용을 같이 다듬자. 이번에는 그냥 복붙시키는 게 아니라, 문단마다 “왜 이걸 써놓는지”도 같이 보면서 가자.

2026. 9. 6. 오후 9:11:36





잠깐! 일단 바탕화면에 CMPE-255 폴더를 만들었는데 여기에다가 touch 명령어들 하면되는거야?

2026. 9. 6. 오후 9:14:31



응 ㅎㅎ 그 **CMPE-255** 폴더를 네 로컬 작업 폴더로 쓸 거면 거기서 하면 돼.

먼저 터미널에서 바탕화면의 그 폴더로 이동해:

```
cd ~/Desktop/CMPE-255
```

현재 위치 확인:

```
pwd
```

출력이 대충

```
/Users/너의계정/Desktop/CMPE-255
```

처럼 나오면 맞아.

그다음 여기 안에 우리가 작업할 폴더를 만들자. 폴더 이름을 Assignment 1로 쓰고 싶으면:

```
mkdir -p "Assignment 1/Part-1/house-prices"
```

그리고 이동:

```
cd "Assignment 1/Part-1/house-prices"
```

여기서 이제:

```
touch intent.md
```

```
touch spec.md
```

```
touch plan.md
```

하면 돼.

그리고 하위 폴더도:

```
mkdir -p notebooks data figures results chat-transcript report
```

확인하려면:

```
ls
```

그러면 대충 이렇게 보여야 해:

```
chat-transcript
data
figures
intent.md
notebooks
plan.md
report
results
spec.md
```

근데 한 가지 중요한 점이 있어.

이 **CMPE-255** 폴더가 아직 **GitHub repo**를 **clone**한 폴더가 아니라면, 지금 여기서 파일을 만들어도 GitHub랑 자동 연결되지는 않아.

그래서 다음에 **pwd** 랑 **ls -a** 결과를 보여줘. 내가 이 폴더가 그냥 일반 폴더인지, 이미 **Git repo**인지 바로 봐줄게.

2026. 9. 6. 오후 9:14:35



깃헙에 연결해야해. 그리고 하위 폴더면 어디를 말하는거야?

2026. 9. 6. 오후 9:16:24



응, 먼저 **GitHub repo**랑 로컬 폴더를 연결하는 게 먼저야.

그리고 내가 말한 “하위 폴더”는 그냥 **CMPE-255** 안에 들어가는 폴더들을 말한 거야.

예를 들면:

```
Desktop/
└─ CMPE-255/
 └─ Assignment 1/
 └─ Part-1/
 └─ house-prices/
 ├── notebooks/
 ├── data/
 ├── figures/
 ├── results/
 ├── chat-transcript/
 └─ report/
```

즉 `notebooks`, `data`, `figures` 같은 게 `house-prices`의 하위 폴더야.

지금은 먼저 GitHub 연결부터 하자.

## 1. 바탕화면 CMPE-255로 이동

```
cd ~/Desktop/CMPE-255
```

## 2. 폴더가 비어 있다면 GitHub repo를 여기로 바로 clone

네 repo가 이거니까:

```
git clone https://github.com/ChanghyunKim-sjsu/FA2026_CMPE-255-Data-Mining.git .
```

마지막의 `.`이 중요해.

이건:

“새 폴더 만들지 말고 현재 **CMPE-255** 폴더 안에 바로 **repo** 내용 가져와”

라는 뜻이야.

성공하면:

```
ls
```

했을 때 기존 GitHub 파일들이 보여야 해.

그리고:

```
git status
```

했을 때 대략:

```
On branch main
Your branch is up to date with 'origin/main'.
```

이런 식으로 나오면 **GitHub 연결 완료**야.

그다음 우리가 만들 위치는:

```
mkdir -p "Assignment 1/Part-1/house-prices"
```

그리고:

```
cd "Assignment 1/Part-1/house-prices"
```

여기에서야 비로소:

```
touch intent.md
```

```
touch spec.md
```

```
touch plan.md
```

```
mkdir -p notebooks data figures results chat-transcript report
```

를 하는 거야.

## 한 가지 확인

```
git clone
```

 하기 전에 먼저:

```
ls -la ~/Desktop/CMPE-255
```

해봐.

**CMPE-255** 안이 비어 있으면 바로 **clone**해도 되고, 이미 뭔가 들어있으면 덮어쓰기 문제를 피하려고 방법을 조금 바꿔야 해.

```
ls -la
```

 결과 보여줘. 그다음 한 줄씩 같이 가자.

2026. 9. 6. 오후 9:16:29



```
changhyunkim\@Mac CMPE-255 % git clone
[https://github.com/ChanghyunKim-sjsu/FA2026_CMPE-
255-Data-Mining.git](https://github.com/ChanghyunKim-
sjsu/FA2026_CMPE-255-Data-Mining.git) .
fatal: destination path '.' already exists and is not an
empty directory.
```

2026. 9. 6. 오후 9:17:23



응, 에러 원인은 간단해 ㅎㅎ

마지막에 `.` 을 붙였기 때문에 Git이 **현재 CMPE-255** 폴더 안에 바로 **clone**하려고 했는데, 우리가 이미 만  
든 **Assignment 1** 같은 파일/폴더가 있어서:

“현재 폴더가 비어 있지 않아서 clone할 수 없음”

이라고 한 거야.

지금 삭제하거나 다시 만들지 말자. 먼저 안에 뭐가 있는지만 확인하자.

현재 **CMPE-255** 위치에서:

```
ls -la
```

결과를 그대로 보여줘.

아마 이런 식일 가능성이 높아:

```
.
..
Assignment 1
```

그렇다면 네가 만든 **Assignment 1** 은 살려두고, GitHub repo와 안전하게 합치는 방법으로 갈게.

그리고 참고로 네가 입력할 때 URL은 Markdown 형식 말고 **그냥 URL 자체**를 써야 해:

```
git clone https://github.com/ChanghyunKim-sjsu/FA2026_CMPE-255-Data-Mining.git .
```

하지만 지금은 다시 실행하지 말고 `ls -la` 만 먼저 해줘. 결과 보고 다음 한 줄 알려줄게.

2026. 9. 6. 오후 9:17:33



```
changhyunkim\@Mac CMPE-255 % ls -la
total 16
drwxr-xr-x 5 changhyunkim staff 160 Sep 6 21:14 .
drwx-----@ 173 changhyunkim staff 5536 Sep 6 21:15 ..
-rw-r--r--@ 1 changhyunkim staff 6148 Sep 6 21:15
.DS_Store
drwxr-xr-x 4 changhyunkim staff 128 Sep 6 21:15
Assignment 1
drwxr-xr-x 63 changhyunkim staff 2016 Sep 5 22:16
HW2
```

2026. 9. 6. 오후 9:18:08



## Thinking

- 시작 위치 확인하기

현재 폴더에 기존 과제가 있으므로, 먼저 원격 저장소를 별도 폴더에 복제하는 편이 안전해요. 복제 후 Assignment 1과 HW2를 비교해 충돌 없이 정리하겠습니다.

- 시작 위치를 확인함

✓ 6s 동안 처리함

좋아, 원인 완전히 확인됐어 ㅎㅎ

현재 **CMPE-255** 는 그냥 네 로컬 수업 폴더고, 안에 이미:

CMPE-255/

├─ Assignment 1/

└─ HW2/

가 있어서 여기에 repo를 바로 clone하면 안 돼.



가장 안전한 방법은 **GitHub repo**를 **CMPE-255** 안의 별도 폴더로 **clone**한 뒤, 우리가 만든 **Assignment 1**을 그 repo 안으로 옮기는 거야.

지금 **CMPE-255** 위치에서 이것만 실행해:

```
git clone https://github.com/ChanghyunKim-sjsu/FA2026_CMPE-255-Data-Mining.git
```

이번엔 마지막에 **.** 없음.

성공하면 구조가 이렇게 될 거야:

```
CMPE-255/
├─ Assignment 1/ ← 방금 우리가 만든 로컬 폴더
├─ HW2/
└─ FA2026_CMPE-255-Data-Mining/ ← GitHub와 연결된 진짜 repo
```

그다음:

```
cd FA2026_CMPE-255-Data-Mining
```

그리고 확인:

```
git status
```

정상이라면 대략:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

라고 나올 거야.

**일단 여기까지만 해봐.**

성공하면 그 다음에 우리가 만든 바깥쪽 **Assignment 1**을 GitHub repo 안으로 안전하게 옮기자. 기존 GitHub 파일이랑 충돌 안 나는지도 먼저 확인하고.

2026. 9. 6. 오후 9:18:17



아 잠깐만 폴더 이름이 헛갈렸다 CMPE-272로 해야해

2026. 9. 6. 오후 9:18:41



아 ㅋㅋ 잠깐, 여기서는 그렇게 바꾸면 오히려 더 헛갈려.

지금 우리가 다시 하는 과제는 **CMPE-255 Data Mining - Assignment 1**이고, GitHub repo도:

`FA2026_CMPE-255-Data-Mining`

이거잖아.

그래서 로컬 폴더도 **CMPE-255**로 두는 게 맞아.

`CMPE-272` 는 네 Enterprise Software Platforms 수업 쪽이고.

현재 구조:

```
Desktop/
├── CMPE-255/
│ ├── Assignment 1/
│ └── HW2/
```

이게 오히려 정상적인 구조야.

그러니까 폴더 이름은 지금 `CMPE-255` 그대로 두자. 삭제하거나 **rename**할 필요 없어.

우리가 해야 할 건 이제 그 안에 GitHub repo를 clone하는 것뿐이야:

```
cd ~/Desktop/CMPE-255
```

그리고:

```
git clone https://github.com/ChanghyunKim-sjsu/FA2026_CMPE-255-Data-Mining.git
```

그러면:

```
CMPE-255/
├── Assignment 1/
```

← 우리가 방금 만든 임시 작업

└─ HW2/  
└─ FA2026\_CMPE-255-Data-Mining/ ← GitHub repo

가 돼.

**272로 바꾸기 전에 잘 말해줬어 ㅋㅋ 지금 과제는 255가 맞아.**

*2026. 9. 6. 오후 9:18:49*



```
changhyunkim\@Mac CMPE-255 % ls -la
total 16
drwxr-xr-x 4 changhyunkim staff 128 Sep 6 21:20 .
drwx-----@ 174 changhyunkim staff 5568 Sep 6 21:19 ..
-rw-r--r--@ 1 changhyunkim staff 6148 Sep 6 21:20 .DS_Store
drwxr-xr-x 4 changhyunkim staff 128 Sep 6 21:15 Assignment 1
```

2026. 9. 6. 오후 9:20:36



좋아. 이제 깔끔해졌어 ㅎㅎ

현재 CMPE-255 안에는:

```
CMPE-255/
└─ Assignment 1/
```

만 있는 상태네.

그러면 여기서 이제 GitHub repo를 clone하면 돼.

현재 위치가 이미 CMPE-255 니까:

```
git clone https://github.com/ChanghyunKim-sjsu/FA2026_CMPE-255-Data-Mining.git
```

실행하고,

```
ls
```

하면 아마:

```
Assignment 1
FA2026_CMPE-255-Data-Mining
```

이렇게 나올 거야.

그다음 repo 안으로 들어가:

```
cd FA2026_CMPE-255-Data-Mining
```

그리고:

```
git status
```

확인해줘.

정상이라면 `working tree clean` 비슷하게 나올 거야.

그 다음 단계에서 **바깥의** `Assignment 1` 폴더를 **repo 안으로 옮길지**, 아니면 repo 안에 새로 `Assignment 1` 을 만들지 결정하자.

지금은 일단 clone까지만 해보자.

2026. 9. 6. 오후 9:20:40



Clone 했어. 근데 아까 만들어둔 폴더들 in Assignemtn 1 다시 깃  
헙 클론한 데로 옮겨야겠는걸>

2026. 9. 6. 오후 9:22:13



## Thinking

- 정리하며 이동하기

먼저 저장소 안에 같은 이름의 폴더가 있는지 확인하고 있어요. 저장소 폴더에서 `ls -la`를 실행한 뒤, 충돌이 없다면 상위 폴더의 `Assignment 1`을 안전하게 이동하면 됩니다.

- 정리하며 이동함

✓ 11s 동안 처리함

맞아 ㅎㅎ 이제 아까 바깥에 만들어둔 `Assignment 1`을 **GitHub clone** 폴더 안으로 가져와야 해.

다만 바로 `mv` 하지는 말자. 우리가 Work에서 이미 GitHub에 `Assignment-1`을 만들어놨기 때문에, 지금 repo 안에 비슷한 폴더가 이미 있을 가능성이 높아. `Assignment-1`과 `Assignment 1`이 동시에 생기면 나중에 더 헷갈려.

지금 clone한 repo 안으로 들어가서:

```
cd ~/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining
```

그리고 이것만 실행해줘:

```
ls -la
```

특히 출력에

```
Assignment-1
```

또는

## Assignment 1

이 있는지 볼 거야.

현재 구조는 대략:

```
Desktop/
├── CMPE-255/
│ ├── Assignment 1/ ← 우리가 방금 만든 새 작업 폴더
│ │
│ ├── FA2026_CMPE-255-Data-Mining/ ← GitHub clone
│ │ ├── ...
│ │ └── Assignment-1/ ← 아마 Work가 만든 기존 폴더
```

일 가능성이 커.

`ls -la` 결과 보여줘.

그러면 내가 보고

- 기존 `Assignment-1` 을 살릴지
- `Assignment 1` 로 통일할지
- 우리가 만든 새 파일만 안전하게 옮길지

정확한 `mv` 명령어 한 줄씩 줄게.

지금은 실수로 덮어쓰는 것만 피하면 돼.

2026. 9. 6. 오후 9:22:23



```
changhyunkim\@Mac CMPE-255 % ls -la
total 16
drwxr-xr-x 4 changhyunkim staff 128 Sep 6 21:22 .
drwx-----@ 174 changhyunkim staff 5568 Sep 6 21:19 ..
-rw-r--r--@ 1 changhyunkim staff 6148 Sep 6 21:22 .DS_Store
drwxr-xr-x 7 changhyunkim staff 224 Sep 6 21:21 FA2026_CMPE-255-Data-Mining
```

2026. 9. 6. 오후 9:23:04



오케이, 지금 출력 보니까 **바깥쪽에 있던 Assignment 1 폴더는 현재 안 보여.**

지금 **CMPE-255** 안에는 clone한 repo만 있어:

```
CMPE-255/
└── FA2026_CMPE-255-Data-Mining/
```

그러니까 아까 만든 **Assignment 1** 이 이미 지워졌거나, 다른 위치로 옮겨졌을 가능성이 있어.

일단 repo 안으로 들어가서 확인하자:

```
cd FA2026_CMPE-255-Data-Mining
```

그리고:

```
ls -la
```

그 결과 보여줘.

거기 안에 아마 Work가 만들어둔:

```
Assignment-1
```

이 있을 가능성이 커.



그럼 굳이 아까 만든 폴더를 옮길 필요 없이, **clone된 repo** 안의 **Assignment-1** 을 그대로 작업 베이스로 쓰면 돼.

우선 **ls -la** 결과만 보여줘.

2026. 9. 6. 오후 9:23:13



응 내가 지워버리고 클론한 폴더에 있는 Assignment 1으로 다 옮겼어

2026. 9. 6. 오후 9:23:31



좋아, 그럼 이제 제일 깔끔한 상태야 ㅎㅎ

앞으로는 **clone된 repo** 안의 Assignment 1 만 작업본으로 쓰자.

지금 먼저 repo 안에서 위치 확인해:

```
pwd
```

대충 이렇게 나와야 해:

```
/Users/changhyunkim/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining
```

그리고:

```
ls -la
```

해서 Assignment 1 이 보이면 됨.

그다음:

```
cd "Assignment 1"
```

그리고 구조 확인:

```
find . -maxdepth 3 -type d | sort
```

파일도 같이 보려면:

```
find . -maxdepth 3 -type f | sort
```

이 두 결과를 보여줘.

내가 보고 **README/DESIGN**은 유지하고, 우리가 새로 만든 **Part 1** 골격이 제대로 들어갔는지 바로 체크해줄게.

그 다음엔 `intent.md`, `spec.md`, `plan.md` 부터 실제 내용 채우고 첫 commit 하자.

2026. 9. 6. 오후 9:23:36



```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
ls -la
total 8
drwxr-xr-x 7 changhyunkim staff 224 Sep 6 21:21 .
drwxr-xr-x 4 changhyunkim staff 128 Sep 6 21:22 ..
drwxr-xr-x 12 changhyunkim staff 384 Sep 6 21:21 .git
drwxr-xr-x 3 changhyunkim staff 96 Sep 6 21:21
Assignment 1
drwxr-xr-x 10 changhyunkim staff 320 Sep 6 21:21
Assignment-1
drwxr-xr-x 13 changhyunkim staff 416 Sep 6 21:22
Assignment1
-rw-r--r-- 1 changhyunkim staff 410 Sep 6 21:21
README.md
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
Assignment-1랑 Assignment1 은 백업으로 남겨두자
```

2026. 9. 6. 오후 9:24:11



응, 그게 제일 안전해. 🍌

지금 구조는:

```
FA2026_CMPE-255-Data-Mining/
├─ Assignment 1/ ← 앞으로 우리가 작업할 최종본
├─ Assignment-1/ ← 백업
├─ Assignment1/ ← 백업
└─ README.md
```

이렇게 두고, **앞으로는** Assignment 1 만 수정하자.

먼저 그 폴더로 들어가:

```
cd "Assignment 1"
```

그리고 현재 안에 뭐가 있는지 확인하자:



```
find . -maxdepth 4 -print | sort
```

이 결과를 보여줘.

내가 그걸 보고 바로:

- README
- DESIGN
- Part-1/house-prices
- intent.md
- spec.md
- plan.md
- 하위 폴더들

이 제대로 들어있는지 확인해줄게.

그리고 백업 폴더들은 **지금은 절대 건드리지 말자**. 최종 제출 다 끝난 뒤에 정리해도 늦지 않아.

2026. 9. 6. 오후 9:24:14



```
changhyunkim\@Mac Assignment 1 % find . -maxdepth 4
-print | sort
.
./README.md
```

2026. 9. 6. 오후 9:24:34



오케이, 지금 Assignment 1 은 거의 빈 폴더네 ㅎㅎ

그리고 파일명이 README.md 로 오타가 있어. 이걸 먼저 README.md 로 고치자.

```
mv README.md README.md
```

그다음 우리가 사용할 기본 구조를 만들자:

```
mkdir -p Part-1/house-prices/{notebooks,data,figures,results,chat-
transcript,report}
mkdir -p Part-2/{customer-segmentation,market-basket-analysis,anomaly-detection}
```

그리고 Part 1의 문서 3개 생성:

```
touch Part-1/house-prices/intent.md
touch Part-1/house-prices/spec.md
touch Part-1/house-prices/plan.md
```

빈 폴더들도 Git이 추적할 수 있게:

```
touch Part-1/house-prices/notebooks/.gitkeep
touch Part-1/house-prices/data/.gitkeep
touch Part-1/house-prices/figures/.gitkeep
touch Part-1/house-prices/results/.gitkeep
touch Part-1/house-prices/chat-transcript/.gitkeep
touch Part-1/house-prices/report/.gitkeep
```

그리고 구조 확인:

```
find . -maxdepth 4 -print | sort
```

정상이면 대략 이렇게 보여야 해:

```
Assignment 1/
├─ README.md
├─ Part-1/
│ └─ house-prices/
│ ├── intent.md
│ ├── spec.md
│ ├── plan.md
│ ├── notebooks/
│ ├── data/
│ ├── figures/
│ ├── results/
│ ├── chat-transcript/
│ └─ report/
└─ Part-2/
 ├── customer-segmentation/
 ├── market-basket-analysis/
 └─ anomaly-detection/
```

그리고 **DESIGN** 파일은 지금 이 새 폴더에는 없는 것 같아. 기존 백업에 있는 걸 나중에 가져올지, 아니면 이번에는 **intent/spec/plan** 으로 대체할지 결정하면 돼.

일단 위 명령들 실행하고 **find** 결과 보여줘.

그러면 바로 **intent.md** 내용부터 같이 채우자.

2026. 9. 6. 오후 9:24:41



```
changhyunkim\@Mac Assignment 1 % find . -maxdepth 4
-print | sort
.
./Part-1
./Part-1/house-prices
./Part-1/house-prices/chat-transcript
./Part-1/house-prices/chat-transcript/.gitkeep
./Part-1/house-prices/data
./Part-1/house-prices/data/.gitkeep
./Part-1/house-prices/figures
./Part-1/house-prices/figures/.gitkeep
./Part-1/house-prices/intent.md
./Part-1/house-prices/notebooks
./Part-1/house-prices/notebooks/.gitkeep
./Part-1/house-prices/plan.md
./Part-1/house-prices/report
./Part-1/house-prices/report/.gitkeep
./Part-1/house-prices/results
./Part-1/house-prices/results/.gitkeep
./Part-1/house-prices/spec.md
./Part-2
./Part-2/anomaly-detection
./Part-2/customer-segmentation
./Part-2/market-basket-analysis
./README.md
```

2026. 9. 6. 오후 9:25:53



완벽해 ㅎㅎ **Part 1** 골격은 제대로 만들어졌어. 이제 이 Assignment 1 폴더만 앞으로 작업하면 돼.

한 가지만 먼저 처리하자. 지금 Part 2의 세 폴더는 비어 있어서 **Git**은 빈 폴더를 **GitHub**에 올리진 않아.

그래서:

```
touch Part-2/customer-segmentation/.gitkeep
```

```
touch Part-2/market-basket-analysis/.gitkeep
```





`touch Part-2/anomaly-detection/.gitkeep`

해두자.

그 다음부터는 진짜 문서 내용을 채우자. **한꺼번에 하지 말고** `intent.md` 부터.

현재 `Assignment 1` 위치에서:

```
nano Part-1/house-prices/intent.md
```

그리고 아래 내용을 넣어줘.

```
Part 1 – Project Intent
```

```
Project Title
```

```
AI-Assisted House Price Prediction
```

```
Objective
```

```
The goal of this project is to build an end-to-end data science workflow
that predicts residential house prices using the Kaggle House Prices dataset.
```

```
An AI coding assistant will be used throughout the data science lifecycle
for planning, coding, debugging, analysis, and documentation.
```

```
However, AI-generated outputs will be reviewed, validated, and explained
in my own words.
```

```
Business Problem
```

```
House prices depend on many factors such as property quality, living area,
age, garage capacity, neighborhood, and other characteristics.
```

```
The objective is to determine whether these available property features
can be used to predict a home's sale price accurately.
```

```
Dataset
```

## Kaggle House Prices: Advanced Regression Techniques

Target variable:

- `SalePrice`

The dataset contains numerical and categorical features describing residential properties.

### ## Main Question

Can we build a machine learning model that predicts `SalePrice` from the available property characteristics with reasonable accuracy?

### ## Data Science Approach

The project will follow a structured workflow inspired by CRISP-DM:

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Interpretation and Communication

### ## Role of AI

ChatGPT and/or an agentic coding assistant may help with:

- planning the analysis
- generating and reviewing Python code
- suggesting preprocessing approaches
- suggesting machine learning models
- debugging
- interpreting model results
- organizing project documentation

AI-generated results will not be accepted blindly.

The results will be checked and paraphrased based on my own understanding.

```
Success Criteria
```

The project will be considered successful if:

- the dataset is explored and understood
- missing and categorical data are handled appropriately
- multiple regression approaches are evaluated
- model performance is measured using appropriate metrics
- the final model is compared against a baseline
- important features and prediction errors are interpreted
- the entire workflow is reproducible
- the final results can be clearly explained in my own words

nano 저장은:

```
Control + O
Enter
Control + X
```

그리고 확인:

```
cat Part-1/house-prices/intent.md
```

## 이 파일이 하는 역할

이게 아까 이미지에서 본 `**intent.md = “우리가 왜 이걸 하는가?”**`야.

아직 여기에는:

- Random Forest가 몇 점 나왔다
- RMSE가 얼마다
- 어떤 feature가 가장 중요하다

같은 결과는 하나도 없어.

일부러 그래.

지금은:

*House Prices를 예측한다 → AI를 사용한다 → 하지만 사람이 검증하고 이해한다 → 어떤 조건이면 성공이라고 볼지 정한다*

까지만 선언한 거야.

특히 교수님의

*“read and understand and provide a paraphrase of the output”*

요구사항을

*AI-generated results will not be accepted blindly.*

로 프로젝트 원칙 자체에 넣은 게 꽤 중요해.

이거 저장한 다음에는 `spec.md`로 넘어가자. `spec.md`부터는 실제로 어떤 **EDA, preprocessing, models, metrics**를 사용할지 설계하게 돼.

2026. 9. 6. 오후 9:26:02



좋아 스펙.엠디로 ㄱㄱ

2026. 9. 6. 오후 9:27:04



좋아, 바로 `spec.md` 가자.

이 파일은 **\*\*“무엇을 어떻게 분석할 것인가”\*\***를 정하는 문서야. 아직 결과값은 넣지 않고, 분석 설계만 적어.

현재 `Assignment 1` 폴더에서:

```
nano Part-1/house-prices/spec.md
```

그리고 아래 내용 넣어줘.

```
Part 1 – Analysis Specification
```

```
1. Problem Type
```

```
This is a supervised machine learning problem.
```

```
Task:
```

```
- Regression
```

```
Target variable:
```

```
- `SalePrice`
```

```
The goal is to predict residential house sale prices using the available numerical and categorical property features.
```

```

```

```
2. Dataset Validation
```

Before modeling, the dataset will be inspected to understand its structure and quality.

The following checks will be performed:

- dataset shape
- column names
- data types
- missing values
- duplicate rows
- target distribution
- numerical and categorical feature counts

The goal of this phase is to identify potential data quality issues before building any model.

---

### ## 3. Exploratory Data Analysis

Exploratory Data Analysis will be used to understand the relationships between the available features and `SalePrice`.

The analysis will include:

- `SalePrice` distribution
- summary statistics
- missing-value analysis
- numerical correlation analysis
- relationships between selected features and `SalePrice`
- potential outlier inspection

Example features to investigate may include:

- `OverallQual`
- `GrLivArea`
- `GarageCars`
- `TotalBsmtSF`
- `YearBuilt`

These are initial candidates only.

The actual analysis will be based on evidence from the dataset.

---

## ## 4. Data Preparation

The feature set contains both numerical and categorical variables, so different preprocessing strategies will be used.

### ### Numerical Features

Possible preprocessing steps:

- missing-value imputation
- optional transformations when justified
- feature scaling when required by the selected model

### ### Categorical Features

Possible preprocessing steps:

- missing-value imputation
- categorical encoding

Scikit-learn pipelines and `ColumnTransformer` will be used when practical to keep preprocessing reproducible and reduce data leakage.

---

## ## 5. Training and Validation Strategy

The dataset will be divided into training and validation sets.

A fixed random state will be used so that results can be reproduced.

Cross-validation may also be used to compare models more reliably.

The validation data will not be used to train the model.

---

## ## 6. Baseline

A simple baseline model will be established before evaluating more advanced machine learning models.

The purpose of the baseline is to answer:

> Does the machine learning model actually perform better than a simple  
> prediction strategy?

Possible baseline:

- `DummyRegressor`

---

## ## 7. Candidate Models

Several regression approaches will be compared.

Initial candidate models include:

1. Dummy Regressor
2. Linear Regression or a regularized linear model
3. Random Forest Regressor
4. Gradient Boosting Regressor

Additional models will only be introduced if they provide a clear analytical benefit.

The final model will be selected based on validation performance and model behavior rather than model complexity alone.

---



## ## 8. Evaluation Metrics

The primary evaluation metrics will be:

### ### RMSE

Root Mean Squared Error measures the typical prediction error while giving larger errors more weight.

Lower values are better.

### ### MAE

Mean Absolute Error measures the average absolute difference between predicted and actual house prices.

Lower values are better.

### ### R<sup>2</sup>

R-squared may be reported as a supporting metric to show how much of the variation in `SalePrice` is explained by the model.

Higher values are generally better.

No final model will be selected based on a single metric alone.

---

## ## 9. Model Interpretation

After the models are evaluated, the final analysis will investigate why the model behaves as it does.

The interpretation phase may include:

- important predictive features
- predicted versus actual values
- large prediction errors

- examples of underprediction and overprediction
- possible limitations of the model

The purpose is not only to produce a prediction score, but also to understand what the model learned.

---

## ## 10. Reproducibility

The final project should be reproducible from the GitHub repository.

The final notebook should:

- run from top to bottom without errors
- use a fixed random state where appropriate
- include the required preprocessing
- generate the reported metrics
- generate the final visualizations

Before submission, the notebook will be restarted and executed using:

> Restart Kernel → Run All

Only results from the final successful run will be used in the README, report, and YouTube walkthrough.

---

## ## 11. Required Artifacts

The final Part 1 project should contain:

- executed Jupyter notebook
- dataset or dataset acquisition instructions
- Python dependency information
- generated figures
- model comparison results
- written interpretation

- README documentation
- AI conversation transcript
- final report if useful
- YouTube walkthrough link

Medium publication is not required.

---

## ## 12. Human Validation of AI Output

AI tools may assist with planning, coding, debugging, and interpretation.

However:

- generated code will be executed and checked
- model results will be independently verified
- unexpected results will be investigated
- final explanations will be paraphrased based on my own understanding

The purpose of the AI assistant is to support the data science workflow, not to replace understanding of the analysis.

저장:

Control + O

Enter

Control + X

그리고 확인:

`cat Part-1/house-prices/spec.md`

여기서 중요한 차이는 이거야:

- `intent.md` = 왜 하는가
- `spec.md` = 무엇을 어떤 기준으로 할 것인가

- 다음 `plan.md` = 그걸 실제 작업 순서로 어떻게 쪼갤 것인가

특히 이번 spec에는 일부러 “**Random Forest가 최고일 것이다**” 같은 결론을 미리 안 넣었어. 실제로 데이터를 돌려보고 결정해야 하니까.

그리고 `DummyRegressor`를 baseline으로 넣은 것도 꽤 좋아. 나중에 우리가 모델 RMSE를 보고 단순히 “25,000 나왔네”가 아니라,

“아무 생각 없이 평균 가격만 예측하는 모델보다 얼마나 좋아졌는가?”

를 비교할 수 있거든.

이거 저장했으면 바로 `plan.md`로 넘어가면 돼.

2026. 9. 6. 오후 9:27:09



플랜 ㄱㄱ

2026. 9. 6. 오후 9:27:50



좋아, `plan.md` 가자. 이걸 이제 실제로 우리가 어떤 순서로 작업할지 체크리스트로 쪼개는 파일이야.

```
nano Part-1/house-prices/plan.md
```

아래 내용 넣어줘.

```
Part 1 – Implementation Plan
```

```
Phase 1 – Project Setup
```

- [ ] Confirm the House Prices dataset is available
- [ ] Place the dataset in the `data/` directory
- [ ] Create the final Jupyter notebook
- [ ] Confirm the Python environment and required packages
- [ ] Record the initial project structure in GitHub

```

```

```
Phase 2 – Data Understanding
```

- [ ] Load `train.csv`
- [ ] Inspect the first rows
- [ ] Check dataset shape
- [ ] Inspect column names and data types
- [ ] Count numerical and categorical features
- [ ] Check duplicate rows
- [ ] Check missing values
- [ ] Inspect `SalePrice` summary statistics
- [ ] Summarize the initial findings in my own words

```

```

## ## Phase 3 – Exploratory Data Analysis

- [ ] Plot the `SalePrice` distribution
- [ ] Investigate whether the target is skewed
- [ ] Analyze missing-value patterns
- [ ] Examine numerical correlations with `SalePrice`
- [ ] Investigate important candidate features
- [ ] Inspect potential outliers
- [ ] Save useful figures to the `figures/` directory
- [ ] Write a short interpretation after each important visualization

Initial relationships to investigate may include:

- `OverallQual` vs `SalePrice`
- `GrLivArea` vs `SalePrice`
- `GarageCars` vs `SalePrice`
- `TotalBsmtSF` vs `SalePrice`
- `YearBuilt` vs `SalePrice`

These may change depending on the actual EDA findings.

---

## ## Phase 4 – Data Preparation

- [ ] Separate features (`X`) and target (`y`)
- [ ] Create training and validation splits
- [ ] Identify numerical columns
- [ ] Identify categorical columns
- [ ] Build numerical missing-value preprocessing
- [ ] Build categorical missing-value preprocessing
- [ ] Encode categorical variables
- [ ] Apply scaling only when justified by the model
- [ ] Combine preprocessing with `ColumnTransformer`
- [ ] Verify the preprocessing pipeline runs successfully

The preprocessing pipeline should be fit only on training data to reduce data leakage.

---

## ## Phase 5 – Baseline Model

- [ ] Train a `DummyRegressor`
- [ ] Generate validation predictions
- [ ] Calculate RMSE
- [ ] Calculate MAE
- [ ] Calculate  $R^2$  if useful
- [ ] Save the baseline results

The baseline will provide a reference point for judging whether the machine learning models provide meaningful improvement.

---

## ## Phase 6 – Candidate Models

Evaluate multiple regression models.

### ### Model 1 – Linear / Regularized Linear Model

- [ ] Train the model
- [ ] Generate validation predictions
- [ ] Calculate evaluation metrics
- [ ] Record the results

### ### Model 2 – Random Forest Regressor

- [ ] Train the model
- [ ] Generate validation predictions
- [ ] Calculate evaluation metrics
- [ ] Record the results

### ### Model 3 – Gradient Boosting Regressor

- [ ] Train the model
- [ ] Generate validation predictions

- [ ] Calculate evaluation metrics
- [ ] Record the results

Additional models should only be added if there is a clear reason.

---

## ## Phase 7 – Model Comparison

- [ ] Combine model metrics into a comparison table
- [ ] Compare RMSE
- [ ] Compare MAE
- [ ] Compare  $R^2$  if reported
- [ ] Compare all models against the baseline
- [ ] Consider cross-validation for stronger validation
- [ ] Select the final model
- [ ] Explain why the final model was selected

The final model should not be selected only because it is the most complex.

---

## ## Phase 8 – Model Interpretation

- [ ] Create predicted vs actual visualization
- [ ] Calculate prediction errors
- [ ] Inspect the largest errors
- [ ] Identify major underpredictions
- [ ] Identify major overpredictions
- [ ] Investigate important predictive features when supported by the model
- [ ] Discuss potential explanations for the observed errors
- [ ] Document model limitations

---

## ## Phase 9 – Final Validation

Before using any result in the final submission:



- [ ] Restart the notebook kernel
- [ ] Run all notebook cells from top to bottom
- [ ] Confirm that no cell fails
- [ ] Confirm reported metrics match notebook output
- [ ] Confirm figures match the final run
- [ ] Confirm random states are fixed where appropriate
- [ ] Remove unused experimental code
- [ ] Remove misleading or outdated results

Only the results from this final successful run will be treated as the official project results.

---

## ## Phase 10 – Documentation

- [ ] Update the Part 1 README
- [ ] Describe the business problem
- [ ] Describe the dataset
- [ ] Explain the workflow
- [ ] Include final model comparison
- [ ] Include important visualizations
- [ ] Explain results in my own words
- [ ] Document limitations
- [ ] Add instructions for reproducing the analysis
- [ ] Record AI-assisted workflow artifacts

---

## ## Phase 11 – Assignment Deliverables

- [ ] Confirm all Part 1 artifacts are in GitHub
- [ ] Export or save the AI conversation transcript
- [ ] Confirm the GitHub repository is publicly viewable
- [ ] Record the Part 1 YouTube walkthrough
- [ ] Explain the end-to-end journey in the video
- [ ] Paraphrase and explain the results rather than reading AI output
- [ ] Add the YouTube URL to the README
- [ ] Perform a final professor-requirement checklist

Medium publication is not required.

저장하고 나오면:

```
cat Part-1/house-prices/plan.md
```

확인하면 돼.

이제 세 문서 역할이 딱 정리됐어:

- `intent.md` → 왜 하는가
- `spec.md` → 무엇을 어떤 기준으로 할 것인가
- `plan.md` → 실제로 어떤 순서로 할 것인가

다음은 문서 작업 끝. 이제 `train.csv` 를 `data/` 에 넣고 진짜 **Phase 1/2** 실행 시작하면 돼.

2026. 9. 6. 오후 9:27:53



77

2026. 9. 6. 오후 9:28:50



좋아. 이제부터는 문서 설계 끝 → 실제 **Part 1** 분석 시작이야. 이번엔 한 단계씩 실행하고 결과를 이해한 다음 넘어가자.

먼저 현재 위치가 Assignment 1 인지 확인:

```
pwd
```

대충 끝이 이렇게 나와야 해:

```
.../FA2026_CMPE-255-Data-Mining/Assignment 1
```

## 1. train.csv 넣기

House Prices의 train.csv 를 여기에 넣자:

```
Assignment 1/
└─ Part-1/
 └─ house-prices/
 └─ data/
 └─ train.csv
```

다운로드 폴더에 있다면 예를 들어:

```
cp ~/Downloads/train.csv Part-1/house-prices/data/train.csv
```

그리고 확인:

```
ls -lh Part-1/house-prices/data
```

train.csv 가 보이면 성공.

---

## 2. Python 환경 만들기

이번 프로젝트 전용 `.venv` 를 `house-prices` 안에 만들자.

```
cd Part-1/house-prices
```

```
python3 -m venv .venv
```

활성화:

```
source .venv/bin/activate
```

그러면 터미널 앞에:

```
(.venv)
```

가 붙어야 해.

그 다음 필요한 기본 패키지만 설치:

```
pip install pandas numpy matplotlib scikit-learn jupyter
```

이번에는 처음부터 라이브러리를 엄청 많이 넣지 않을 거야. 필요해질 때 추가하자.

---

## 3. VS Code 열기

현재 `house-prices` 폴더에서:

```
code .
```

만약 `code` 명령어가 안 되면 VS Code에서 직접:

**File → Open Folder →** `house-prices`

를 열어도 돼.

그리고 `notebooks` 안에 새 파일:

```
house_prices_analysis.ipynb
```

를 만들어.

구조가 이제:

```
house-prices/
├── .venv/
├── data/
│ └── train.csv
├── notebooks/
│ └── house_prices_analysis.ipynb
├── figures/
├── results/
├── chat-transcript/
├── report/
├── intent.md
├── spec.md
└── plan.md
```

이렇게 된다.

---

## 4. Notebook 첫 셀

처음부터 너무 많이 import하지 말자.

첫 번째 Markdown cell:

```
AI-Assisted House Price Prediction
```

```
Assignment 1 – Part 1
```

```
This notebook presents an end-to-end data science workflow using the
Kaggle House Prices: Advanced Regression Techniques dataset.
```

```
The workflow follows a CRISP-DM-inspired process:
```

1. Business Understanding
2. Data Understanding

3. Data Preparation
4. Modeling
5. Evaluation
6. Interpretation

그다음 Code cell:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

실행.

에러 없으면 다음.

---

## 5. 데이터 읽기

중요한 부분이 하나 있어.

Notebook은:

```
house-prices/notebooks/
```

안에 있고 데이터는:

```
house-prices/data/
```

안에 있으니까 상대경로는:

```
../data/train.csv
```

야.

새 Code cell:

```
df = pd.read_csv("../data/train.csv")
```

그리고:

```
df.head()
```

실행.

표가 나오면 우리 프로젝트의 첫 실제 데이터 로딩 성공이야.

---

## Phase 2 — Data Understanding 시작

아직 ML은 안 해.

새 Markdown cell:

```
Data Understanding
```

```
The first step is to inspect the structure and quality of the dataset
before performing preprocessing or modeling.
```

그리고 하나씩 보자.

### Dataset 크기

```
df.shape
```

House Prices 원본이라면 대략:

```
(1460, 81)
```

같은 형태가 나올 거야.

여기서 뜻은:

- 1460 = 집 1,460개
- 81 = column 81개

그리고 이 81개 중 하나가 우리가 예측하려는:

```
SalePrice
```

야.

## Column 확인

```
df.columns.tolist()
```

## 데이터 타입

```
df.dtypes.value_counts()
```

이게 재밌어.

House Prices에는 숫자만 있는 게 아니라:

OverallQual	int
GrLivArea	int
Neighborhood	object
GarageType	object
SalePrice	int

처럼 **numerical + categorical** 데이터가 섞여 있어.

그래서 우리가 `spec.md`에 미리:

*numerical과 categorical을 별도로 preprocessing한다*

라고 써놓은 거야.

## Duplicate 확인

```
df.duplicated().sum()
```

## Missing values 확인



```
missing = df.isnull().sum().sort_values(ascending=False)
```

```
missing[missing > 0].head(20)
```

## Target 확인

```
df["SalePrice"].describe()
```

여기까지만 먼저 하자.

## 지금 목표

아직 Random Forest도 안 하고 Linear Regression도 안 해.

현재 질문은 딱 다섯 개야:

데이터가 몇 행/몇 열인가?

숫자와 category가 얼마나 섞여 있는가?

중복이 있는가?

missing data가 어디에 많은가?

SalePrice 가 대충 어떤 범위인가?

이게 **Data Understanding**이야.

train.csv 넣고 notebook에서 위 코드들을 실행한 다음, df.shape 부터 SalePrice.describe() 까지 나온 결과를 보여줘.

그 결과를 우리가 같이 읽고 네 말로 첫 번째 **Data Understanding interpretation**을 작성하자.

2026. 9. 6. 오후 9:29:01



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

여기까진 했는데, Colab에서 하는것 처럼 실행을 해볼수는 없는거야?

2026. 9. 6. 오후 9:35:40



응, 당연히 가능해. 지금 `.ipynb` 파일로 작업 중이면 **VS Code**에서도 **Colab**처럼 **셀 단위 실행**할 수 있어.

먼저 확인할 것 두 가지가 있어.

1. 파일이 진짜 `house_prices_analysis.ipynb` 인지
2. VS Code 오른쪽 위나 셀 왼쪽에 ▶ 실행 버튼이 보이는지

보이면 이 셀:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

왼쪽의 ▶ 버튼 누르거나, 셀 안에서:

Shift + Enter

하면 실행돼.

만약 실행하려는데 `Select Kernel` 같은 게 뜨면, 아까 만든 가상환경 `.venv` 를 선택하면 돼. 보통:

Python Environments  
→ `.venv`

이런 식으로 보여.

그리고 만약 `.venv` 가 안 보이면 터미널에서 현재 `house-prices` 폴더에서:

```
source .venv/bin/activate
```

한 뒤 VS Code를 다시 열거나:

```
code .
```

해보면 돼.

실행이 잘 되면 import 셀은 출력이 없는 게 정상이고, 다음 셀에:

```
print("Libraries loaded successfully")
```

라고 넣어서 실행해보면 바로 확인할 수 있어.

또는:

```
pd.__version__
```

실행하면 pandas 버전이 출력돼.

즉 **Colab**처럼 완전히 셀 단위로 실행 가능해. 차이는 Colab은 클라우드 환경이고, 지금 VS Code notebook은 네 Mac의 `.venv` Python을 kernel로 쓰는 거야.

지금 화면에 `Select Kernel` 이 뜨는지, 아니면 ▶ 버튼은 있는데 실행이 안 되는지 알려줘. 그 상태 기준으로 바로 다음 단계 알려줄게.

2026. 9. 6. 오후 9:35:43



**\*\*Running cells with 'Python 3.13.0' requires the ipykernel package. 라고 뜨는데?\*\***

2026. 9. 6. 오후 9:38:12



응, 이건 흔한 에러야. 지금 VS Code가 선택한 Python 환경에 `ipykernel` 이 없어서 **notebook cell** 을 못 돌리는 상태야.

우리가 만든 `.venv` 안에 `ipykernel` 만 설치하면 돼.

지금 VS Code 터미널에서 `house-prices` 폴더에 있는지 확인하고:

```
pwd
```

그다음 가상환경 활성화:

```
source .venv/bin/activate
```

터미널 앞에 이렇게 붙으면 돼:

```
(.venv) changhyunkim@Mac house-prices %
```

그리고:

```
python -m pip install ipykernel
```

설치 끝나면 VS Code notebook 오른쪽 위의 **Select Kernel** 클릭해서:

```
Python Environments
→ .venv
```

를 선택해.

가능하면 `Python 3.13.0` 이라고만 적힌 시스템 Python 말고, 경로가 대략:

```
.../house-prices/.env/bin/python
```

인 걸 선택하면 돼.

그 후 다시:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

를 ▶ 또는 **Shift + Enter** 로 실행해봐.

```
Libraries loaded successfully!
```

가 나오면 **Colab**처럼 로컬 **Jupyter Notebook** 실행 환경 준비 완료야.

참고로 구조는 지금 이렇게 되는 거야:

```
VS Code Notebook
 ↓
ipykernel
 ↓
.venv의 Python
 ↓
pandas / numpy / matplotlib
```

즉 **ipykernel** 이 **VS Code notebook** 셀과 **Python** 실행 환경 사이를 연결해주는 역할이라고 보면 돼.

2026. 9. 6. 오후 9:38:18



Text 파일은 이제 실행되는데 파이썬 코드들이 안된다

2026. 9. 6. 오후 9:42:43



아 그럼 지금 **Markdown/Text** 셀은 보이는데 **Python kernel**이 아직 제대로 연결되지 않은 상태일 가능성이 높아. Text 셀은 Python 없이도 렌더링되거든.

지금 이 순서대로 확인해보자.

VS Code 터미널에서 `house-prices` 폴더로 간 다음:

```
pwd
```

끝이 대략 이렇게 되어 있어야 해.

```
.../Assignment 1/Part-1/house-prices
```

그다음 가상환경 활성화:

```
source .venv/bin/activate
```

앞에 `(.venv)` 가 생긴 걸 확인하고:

```
which python
```

결과가 반드시 비슷하게:

```
.../house-prices/.venv/bin/python
```

이어야 해.

이제 필요한 것들이 **이 가상환경 안에** 있는지 확실하게 설치하자.

```
python -m pip install ipykernel pandas numpy matplotlib scikit-learn jupyter
```

그리고 kernel도 명시적으로 등록해버리자:

```
python -m ipykernel install --user --name cmpe255-houseprices --display-name
"CMPE 255 House Prices"
```

성공하면 VS Code notebook 오른쪽 위에 있는 **Select Kernel** 클릭해서:

```
Jupyter Kernel
→ CMPE 255 House Prices
```

를 선택해.

그 다음 아주 간단한 코드 셀부터 테스트:

```
print("Hello CMPE 255!")
```

이게 실행되면 다음:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

까지 실행해.

## 여기서 핵심

지금 VS Code가 아마:

```
Python 3.13.0
```

이라는 **Mac의 일반 Python**을 잡고 있었던 것 같아.

우리가 원하는 건:

```
CMPE 255 House Prices
```

즉:

```
.venv/bin/python
```

이야.

Markdown 셀:

VS Code 자체가 표시 가능

Python 코드 셀:

VS Code

↓

Jupyter / ipykernel

↓

.venv Python

↓

pandas / numpy / matplotlib

라서 kernel이 잘못 잡혀 있으면 **Text**는 되는데 **Python**만 안 되는 현상이 정확히 생겨.

일단 `which python` 결과하고, `print("Hello CMPE 255!")` 실행했을 때 나오는 에러 메시지 그대로 보여줘. 그러면 바로 다음 한 단계만 잡아줄게.

2026. 9. 6. 오후 9:42:51





```
print("Hello CMPE 255!") 실행되는데
```

```

```

```
ModuleNotFoundError Traceback (most
recent call last)
```

```
Cell In[9], line 1
```

```
\----> 1 import pandas as pd
```

```
2 import numpy as np
```

```
3 import matplotlib.pyplot as plt
```

```
4
```

```
ModuleNotFoundError: No module named 'pandas'
```

2026. 9. 6. 오후 9:45:13



좋아, 이제 원인이 확실해졌어.

`print("Hello CMPE 255!")` 는 실행되니까 **kernel** 자체는 정상이야.

문제는 지금 선택한 그 kernel 환경에 **pandas / numpy / matplotlib**이 설치되어 있지 않은 것뿐이야.

VS Code 터미널에서 먼저 가상환경을 활성화해:

```
source .venv/bin/activate
```

앞에 `(.venv)` 가 보이면:

```
python -m pip install pandas numpy matplotlib scikit-learn jupyter ipykernel
```

설치 끝난 뒤 notebook으로 돌아가서 **kernel**을 한 번 **Restart**해.

그리고 다시:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

실행하면 돼.

만약 그래도 같은 에러가 나오면, 그건 터미널의 `.venv` 와 **notebook**이 서로 다른 **Python**을 쓰고 있다는 뜻이야. 그때 notebook 셀에 이걸 실행해봐:

```
import sys
print(sys.executable)
```

출력이 이런 식이어야 해:

```
.../house-prices/.venv/bin/python
```

다른 경로가 나오면 kernel만 다시 맞춰주면 돼.

지금은 일단 `pip install` 부터 해보자.

2026. 9. 6. 오후 9:45:17



아 다른경로가 나오네

2026. 9. 6. 오후 9:46:10



오케이, 그럼 딱 원인 잡혔어 ㅎㅎ

**VS Code notebook**이 네 `.venv`가 아니라 다른 **Python**을 **kernel**로 쓰고 있는 거야.

이제 그걸 바꾸면 돼.

VS Code notebook 오른쪽 위에서 현재 kernel 이름 클릭해. 아마 `Python 3.13.0` 같은 게 보일 거야.

그다음:

```
Select Another Kernel
→ Python Environments
```

로 들어가서 경로가 이런 걸 선택해:

```
.../Assignment 1/Part-1/house-prices/.venv/bin/python
```

즉 핵심은 `.venv/bin/python` 이야.

선택한 다음 notebook 셀에서 다시:

```
import sys
print(sys.executable)
```

실행해봐.

정상이면 대략:

```
/Users/changhyunkim/.../Assignment 1/Part-1/house-prices/.venv/bin/python
```

이렇게 나와야 해.

그다음:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

실행하면 되고.

만약 VS Code의 kernel 목록에 `.venv`가 아예 안 보이면, 터미널에서 `house-prices` 폴더에 있는 상태로:

```
source .venv/bin/activate
```

그리고:

```
python -m pip install ipykernel pandas numpy matplotlib scikit-learn
```

그 다음:

```
python -m ipykernel install --user \
 --name cmpe255-houseprices \
 --display-name "CMPE 255 House Prices"
```

해주면 VS Code kernel 목록에:

```
CMPE 255 House Prices
```

가 생겨.

그걸 선택하면 돼.

지금 `sys.executable`에서 나온 다른 경로를 그대로 보여줘도 돼. 내가 어느 Python을 잡고 있는지 바로 봐줄게.

2026. 9. 6. 오후 9:46:14



```
/Users/changhyunkim/Documents/CMPE-272-Enterprise-Software-Platforms/Assignment 2/.venv/bin/python
라고 뜨는데?
```

2026. 9. 6. 오후 9:47:53



응, 원인 100% 확실해졌어 ㅋㅋ

지금 CMPE-255 notebook이 \*\*예전 CMPE-272 Assignment 2의 `.venv` \*\*를 kernel로 잡고 있어.

즉 지금 상태는:

```
CMPE-255 notebook
↓
CMPE-272 Assignment 2/.venv
```

이렇게 잘못 연결된 거야.

우리가 원하는 건:

```
CMPE-255 notebook
↓
Assignment 1/Part-1/house-prices/.venv
```

이거야.

## 이렇게 바꾸면 돼

먼저 VS Code 터미널에서 현재 `house-prices` 폴더로 이동:

```
cd \~/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/"Assignment 1"/Part-1/house-prices
```

그다음 가상환경 활성화:

```
source .venv/bin/activate
```

그리고 필요한 패키지 설치:

```
python -m pip install ipykernel pandas numpy matplotlib scikit-learn
```

이제 이 `.venv` 를 Jupyter kernel로 등록:

```
python -m ipykernel install --user \
--name cmpe255-houseprices \
--display-name "CMPE 255 House Prices"
```

그다음 VS Code notebook 오른쪽 위 kernel 클릭해서:

```
Select Another Kernel
→ Jupyter Kernel
→ CMPE 255 House Prices
```

선택해.

그리고 이 셀 실행:

```
import sys
print(sys.executable)
```

정상이라면 이제 경로가 대략:

```
/Users/changhyunkim/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/Assignment
1/Part-1/house-prices/.venv/bin/python
```

이렇게 나와야 해.

그 다음:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

실행하면 될 거야.

핵심은 **CMPE-272 kernel**을 버리고 **CMPE-255 전용 kernel**으로 바꾸는 것이야.

이거 하고 `sys.executable` 결과 다시 보여줘.

2026. 9. 6. 오후 9:47:57



URL을 입력하라는데!?

2026. 9. 6. 오후 9:49:39



아 ㅋㅋ 그럼 지금 **“Jupyter Server URL”** 쪽으로 들어간 거야. 그건 우리가 원하는 메뉴가 아니야.

URL은 입력하지 마.

VS Code notebook 오른쪽 위에서 현재 kernel 이름 클릭한 다음, 이렇게 가봐:

```
Select Another Kernel
→ Python Environments
```

또는 VS Code 버전에 따라:

```
Select Kernel
→ Python Environments
```

여기서 **URL 입력하는** Existing Jupyter Server 는 선택하면 안 돼.

우리가 찾는 건 경로가 이런 Python이야:

```
.../Assignment 1/Part-1/house-prices/.venv/bin/python
```

만약 Python Environments 목록에 안 보이면 먼저 터미널에서:

```
cd ~/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/"Assignment 1"/Part-1/house-
prices
source .venv/bin/activate
python -m pip install ipykernel pandas numpy matplotlib scikit-learn
```

그 다음 VS Code에서 **Cmd + Shift + P** 누르고:

```
Python: Select Interpreter
```

검색해서 실행해.



거기서:

```
.venv/bin/python
```

선택.

그 후 notebook으로 돌아와 다시 kernel 선택하면 `.venv` 가 보일 가능성이 높아.

정상 연결됐는지 마지막으로:

```
import sys
print(sys.executable)
```

실행해서 경로에

```
CMPE-255
...
Assignment 1
...
house-prices/.venv/bin/python
```

이 들어가면 성공이야.

**URL 입력 화면에서는 그냥 Esc 눌러서 나오면 돼.**

2026. 9. 6. 오후 9:49:43



뭐지 계속 272 여기에 남아있어

2026. 9. 6. 오후 9:51:55



## Thinking

- 커널 고정 문제 해결하기

VS Code가 이전 프로젝트의 커널을 계속 선택하는 것 같아요. 먼저 VS Code를 완전히 닫고, 새 프로젝트 폴더에서 올바른 가상환경을 활성화한 뒤 커널을 다시 등록해 보세요.

- 커널 고정 문제를 해결함

✓ 9s 동안 처리함

아, 그러면 VS Code가 예전 **272 kernel**을 캐시/고정해서 계속 잡고 있는 것 같아. 이럴 땐 GUI에서 계속 씨름하지 말고, **255 환경을 확실히 등록한 뒤 VS Code**를 새 폴더 기준으로 다시 여는 게 제일 빠르다.

1. VS Code를 완전히 닫아.
2. 터미널에서 정확한 폴더로 이동:

```
cd \~/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/"Assignment 1"/Part-1/house-prices
```

3. 가상환경 활성화:

```
source .venv/bin/activate
```

확인:

```
which python
```

여기 결과가 반드시:

```
.../CMPE-255/.../house-prices/.venv/bin/python
```

이어야 해.

4. 이 환경에 필요한 것 설치:

```
python -m pip install ipykernel pandas numpy matplotlib scikit-learn jupyter
```

5. **255 전용 kernel**을 확실히 등록:

```
python -m ipykernel install --user \
--name cmpe255-houseprices \
--display-name "CMPE 255 House Prices"
```

6. 등록됐는지 확인:

```
jupyter kernelspec list
```

여기에서 최소한:

```
cmpe255-houseprices
```

가 보여야 해.

그리고 아주 중요하게, 이제 이 폴더에서 VS Code를 새로 열자:

```
code .
```

그러면 VS Code가 **CMPE-255** `house-prices` 를 프로젝트 루트로 잡아.

Notebook 열고 오른쪽 위 kernel을 눌러:

```
Select Another Kernel
→ Python Environments
```

→ CMPE 255 House Prices

또는 목록에 바로:

```
CMPE 255 House Prices
```

가 나오면 그걸 선택.

그리고 notebook에:

```
import sys
print(sys.executable)
```

실행해.

우리가 원하는 최종 결과는 정확히 이런 계열이야:

```
/Users/changhyunkim/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/Assignment
1/Part-1/house-prices/.venv/bin/python
```

여기까지 했는데도 **272** 경로가 뜨면, 그때는 notebook 자체나 VS Code 설정에 272 interpreter가 박혀 있는 거니까 그걸 찾아서 지우면 돼.

일단 지금 `jupyter kernelspec list` 결과를 나한테 보여줘. 거기서 **255 kernel**이 실제로 등록됐는지 부터 확인하자.

2026. 9. 6. 오후 9:52:04



```
(.venv) changhyunkim\@Mac house-prices % jupyter
kernel spec list
Available kernels:
 python3 /Users/changhyunkim/Desktop/CMPE-
255/FA2026_CMPE-255-Data-Mining/Assignment 1/Part-
1/house-prices/.venv/share/jupyter/kernels/python3
 cmpe255-houseprices
/Users/changhyunkim/Library/Jupyter/kernels/cmpe255-
houseprices
```

2026. 9. 6. 오후 9:53:05



좋아, 이건 **255 커널이 제대로 등록된 상태야**. 🍌

지금 중요한 건 두 개 다 255 쪽이라는 거야:

python3

→ .../CMPE-255/.../house-prices/.venv/share/jupyter/kernels/python3

cmpe255-houseprices

→ \~/Library/Jupyter/kernels/cmpe255-houseprices

그러니까 문제는 **VS Code notebook**이 아직 예전 **272** 커널을 선택하고 있는 것뿐이야.

이제 VS Code에서 notebook 오른쪽 위 kernel 클릭하고:

Select Another Kernel

→ Jupyter Kernel

→ cmpe255-houseprices

또는 이름이:

CMPE 255 House Prices

로 보이면 그걸 선택해.

선택 후 바로 이 셀 실행:

```
import sys
print(sys.executable)
```

이제 경로가:

```
/Users/changhyunkim/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/Assignment
1/Part-1/house-prices/.venv/bin/python
```

계열로 나오면 완전히 성공이야.

만약 목록에 `CMPE 255 House Prices`가 안 보이면 VS Code에서:

```
Cmd + Shift + P
→ Developer: Reload Window
```

한 번 해주고 다시 kernel 선택해봐.

지금 상태면 거의 다 왔어. 커널 등록 자체는 정상이야.

2026. 9. 6. 오후 9:53:12



```
/Users/changhyunkim/Desktop/CMPE-255/FA2026_CMPE-255-Data-Mining/Assignment 1/Part-1/house-prices/.venv/bin/python
```

된거같아

2026. 9. 6. 오후 9:54:29



응, 완전히 제대로 연결됐어. 🎉

지금 notebook이 정확히 이 환경을 쓰고 있어:

```
.../CMPE-255/.../house-prices/.venv/bin/python
```

즉 이제부터는 CMPE-272랑 섞일 일 없어.

다시 이 셀 실행해봐:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

print("Libraries loaded successfully!")
```

정상 출력되면 다음으로 바로 데이터 로딩:

```
df = pd.read_csv("../data/train.csv")
df.head()
```

그리고 이어서:

```
df.shape
```

```
df.dtypes.value_counts()
```

```
df.duplicated().sum()
```

```
missing = df.isnull().sum().sort_values(ascending=False)
missing[missing > 0].head(20)
```

```
df["SalePrice"].describe()
```

여기까지 실행한 결과를 보여줘.

그다음부터는 우리가 **Data Understanding** 결과를 실제로 읽고 네 말로 해석해보자.

2026. 9. 6. 오후 9:54:33





오케이 된다! 근데 그냥 구글 Colab에서 하는게 편했나ㅋㅋ

```
근데
df.dtypes.value_counts()
df.duplicated().sum()
missing = df.isnull().sum().sort_values(ascending=False)
missing[missing > 0].head(20)
df["SalePrice"].describe()
```

다 따로따로 해야해?

2026. 9. 6. 오후 9:56:32



ㅋㅋ Colab이 확실히 초반 세팅은 더 편해.

근데 지금처럼 GitHub 프로젝트 구조랑 `.venv` 까지 잡아놓으면 VS Code 쪽이 과제 제출/재현성/파일 관리에는 더 좋아. 한 번 세팅해두면 이후엔 훨씬 편해져.

그리고 네 질문: 꼭 다 따로따로 실행할 필요는 없어.

한 셀에 묶어도 돼:

```
print("Data types:")
print(df.dtypes.value_counts())

print("\nDuplicate rows:")
print(df.duplicated().sum())

print("\nTop missing values:")
missing = df.isnull().sum().sort_values(ascending=False)
print(missing[missing > 0].head(20))

print("\nSalePrice summary:")
print(df["SalePrice"].describe())
```

이렇게 하면 한 번 실행으로 쭉 나와.



다만 **학습용 + 과제 설명용**으로는 나는 지금은 살짝 나눠서 하는 걸 추천해.

예를 들면:

```
df.dtypes.value_counts()
```

```
df.duplicated().sum()
```

```
missing = df.isnull().sum().sort_values(ascending=False)
missing[missing > 0].head(20)
```

```
df["SalePrice"].describe()
```

왜냐하면 notebook에서 각 결과가 따로 보여서, 나중에 YouTube에서:

*"First, I checked the data types."*

*"Next, I checked duplicate rows."*

*"Then I inspected missing values."*

이렇게 설명하기 편하거든.

특히 Jupyter는 **한 셀의 마지막 expression만 자동 출력**하는 특성이 있어서, 아래처럼 한 셀에 그냥 줄줄 쓰면:

```
df.dtypes.value_counts()
df.duplicated().sum()
df["SalePrice"].describe()
```

보통 **마지막 describe() 결과만 화면에 보일 수 있어.**

그래서 한 셀에 묶고 싶으면 **print()**를 쓰는 거야.

내 추천은 이번 notebook에서는:

- 관련된 것끼리 1셀
- 중요한 visualization은 1셀

- interpretation은 바로 밑 Markdown 셀

정도로 하자.

예를 들면:

[Code Cell]

Dataset structure checks

[Markdown]

What I observed

[Code Cell]

Missing-value analysis

[Markdown]

What I observed

[Code Cell]

Target analysis

[Markdown]

What I observed

이 구조가 제일 깔끔해.

2026. 9. 6. 오후 9:56:36



좋아 다 실행잘됐어

2026. 9. 6. 오후 9:57:53



좋아, 이제 **Data Understanding**의 첫 체크는 완료야. 다음은 결과를 그냥 보고 넘어가지 말고, 교수님 요구대로 우리가 직접 해석해서 **Markdown**으로 남기는 단계가 좋아.

우선 notebook에 아래 셀 하나 추가해줘:

```
print("Dataset shape:", df.shape)

print("\nData type counts:")
print(df.dtypes.value_counts())

print("\nDuplicate rows:", df.duplicated().sum())

print("\nTop missing values:")
missing = df.isnull().sum().sort_values(ascending=False)
print(missing[missing > 0].head(20))

print("\nSalePrice summary:")
print(df["SalePrice"].describe())
```

이 결과를 그대로 여기 붙여줘.

그러면 내가 그걸 보고 우리가 같이 이런 식의 Markdown을 만들 거야:

```
Initial Data Understanding
```

- The dataset contains \_\_\_\_ rows and \_\_\_\_ columns.
- The dataset includes both numerical and categorical variables.
- There are \_\_\_\_ duplicate rows.
- Several features contain substantial missing values, especially \_\_\_\_.
- `SalePrice` ranges from approximately \_\_\_\_ to \_\_\_\_.
- The average sale price is approximately \_\_\_\_.

중요한 건 내가 숫자만 채워주는 게 아니라, **“이 결과가 왜 중요한지”**도 같이 볼 거야.

예를 들어 missing value가 많다고 해서 바로:

*“missing data니까 행 삭제!”*

하면 안 돼.

House Prices에서는 어떤 **NaN** 이 실제로는

**“Garage가 없음”, “Pool이 없음”** 같은 의미일 수도 있거든.

그래서 다음 흐름은:

**Data structure 확인**  → 결과 해석 → **missing values의 의미 조사** → **SalePrice distribution** 시각화

이렇게 가자.

지금 나온 output만 붙여줘.

2026. 9. 6. 오후 9:57:58



```
Dataset shape: (1460, 81) Data type counts: str 43
int64 35 float64 3 Name: count, dtype: int64
Duplicate rows: 0 Top missing values: PoolQC 1453
MiscFeature 1406 Alley 1369 Fence 1179
MasVnrType 872 FireplaceQu 690 LotFrontage
259 GarageYrBlt 81 GarageCond 81 GarageType
81 GarageFinish 81 GarageQual 81 BsmtFinType2
38 BsmtExposure 38
...
50% 163000.000000 75% 214000.000000 max
755000.000000 Name: SalePrice, dtype: float64
```

2026. 9. 6. 오후 9:58:17



## Thinking

- 요약 해석하기

출력상 1,460개 주택과 80개 설명변수, 목표변수 `SalePrice`가 보입니다. `PoolQC`, `MiscFeature`, `Alley`, `Fence` 등은 결측이 매우 많아 삭제나 "없음" 범주화를 검토해야 합니다.

- 요약을 해석했습니다

✓ 8s 동안 처리함

좋아. 지금 결과만 봐도 첫 **Data Understanding**에서 꽤 중요한 특징들이 보였어. 🍌

가장 먼저 구조부터 보면:

- **1,460 rows × 81 columns**
- 그중 `SalePrice`가 target이니까 실제 예측에 사용할 후보 feature는 최대 **80개**
- `str` 43개 + `int64` 35개 + `float64` 3개 → 숫자형과 범주형 데이터가 많이 섞여 있음
- duplicate row는 **0개**
- `SalePrice` 중앙값(median)은 **\$163,000**



- 75% 지점은 **\$214,000**
- 최대값은 **\$755,000**

특히 missing values가 재미있어.

PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
...	

처음 보면:

“와, 데이터가 엄청 지저분하네. 이 column들 지워야 하나?”

싶을 수 있는데 **아직 그러면 안 돼.**

예를 들어 PoolQC 는 이름 자체가 **Pool Quality**잖아. 집에 수영장이 없는 경우 NaN 일 가능성이 높아. 즉 여기서 NaN 은 단순히 “데이터를 잃어버렸다”가 아니라:

## No Pool

이라는 실제 정보를 표현할 수도 있어.

Alley , Fence , FireplaceQu , Garage... , Bsmt... 같은 column도 비슷한 가능성이 있어.

그래서 이게 바로 우리가 아까 spec.md 에서 **missing-value analysis**를 **preprocessing**보다 먼저 하자고 한 이유야.

## Notebook에 이 Markdown 셀을 넣자

방금 코드 output 아래에 **Markdown cell** 하나 만들어서:

```
Initial Data Understanding
```

```
The training dataset contains 1,460 observations and 81 columns.
Since `SalePrice` is the target variable, the remaining columns represent
potential features for predicting house prices.
```

The dataset contains a mixture of numerical and categorical variables. This means that a single preprocessing method cannot be applied to all features, and numerical and categorical columns will need to be handled separately.

No duplicate rows were found in the dataset.

Several variables contain a large number of missing values. Features such as `PoolQC`, `MiscFeature`, `Alley`, and `Fence` are missing for most observations. However, these missing values should not automatically be treated as data-quality errors because some may represent the absence of a property feature, such as a house having no pool, alley access, or fence.

The median `SalePrice` is \$163,000, while the 75th percentile is \$214,000 and the maximum observed price is \$755,000. The large difference between the typical sale price and the maximum suggests that the target variable may be right-skewed and may contain high-price outliers.

Further analysis of the `SalePrice` distribution is therefore needed before modeling.

이건 꽤 좋은 첫 interpretation이야.

그리고 중요한 건 마지막 문장이야:

**“target이 *skewed*된 것 같다 → 그럼 다음에는 실제 *distribution*을 그려보자.”**

이렇게 분석이 앞 단계 결과 → 다음 단계 질문으로 연결되는 게 좋아.

다음: `SalePrice`를 실제로 그려보자

새 Markdown cell:

```
Target Variable Analysis
```

```
Before building a regression model, I examined the distribution of
`SalePrice` to understand the range, skewness, and possible extreme values.
```



그 아래 code cell:

```
plt.figure(figsize=(8, 5))

plt.hist(df["SalePrice"], bins=30, edgecolor="black")

plt.xlabel("Sale Price")
plt.ylabel("Number of Houses")
plt.title("Distribution of SalePrice")

plt.show()
```

그리고 하나 더:

```
df["SalePrice"].skew()
```

이 두 개만 실행해봐.

여기서 우리가 배우려는 건 단순히 “예쁜 **histogram** 만들기”가 아니라, 아까 숫자로 봤을 때 추측했던:

`SalePrice` 가 정말 오른쪽으로 긴 꼬리를 가지고 있는가?

를 데이터로 확인하는 과정이야.

그래프랑 `skew()` 숫자가 나오면 보여줘. 그다음 **skewness**가 뭔지부터 같이 이해하고 넘어가자.

2026. 9. 6. 오후 9:58:27



np.float64(1.8828757597682129)

2026. 9. 6. 오후 10:00:01



좋아. `skewness ≈ 1.88` 이면 **\*\*SalePrice가 꽤 강하게 오른쪽으로 치우쳐 있다(right-skewed)\*\***고 보면 돼.

쉽게 말하면 대부분의 집은 중간 가격대에 몰려 있는데, **아주 비싼 집 몇 채가 오른쪽 꼬리를 길게 끌고 있는 분포**라는 뜻이야. 아까 최대값이 `$755,000` 이었던 것도 이 해석과 잘 맞고.

보통 skewness를 아주 거칠게 보면:

- `0` 근처 → 대칭에 가까움
- `0.5\~1` → 어느 정도 치우침
- `> 1` → 꽤 강한 skew

라서 `1.88` 은 확실히 높은 편이야.

Notebook에는 이렇게 해석을 남기면 좋아:

```
SalePrice Distribution Interpretation
```

```
The `SalePrice` distribution is strongly right-skewed.
```

```
The calculated skewness is approximately 1.88, which indicates that most houses are concentrated in the lower and middle price ranges, while a smaller number of expensive homes create a long right tail.
```

```
This suggests that extreme high-price observations may have a noticeable effect on regression models and evaluation metrics such as RMSE.
```

```
A log transformation of `SalePrice` may be investigated later, but I will first continue exploring the original target distribution and feature relationships before deciding whether the transformation is justified.
```

마지막 문장이 중요해. 지금 당장 `log(SalePrice)`로 바꾸지 말고, **\*\*“필요할 수 있다는 가설만 세워두자”\*\***는 거야.

다음은 내가 보기엔 **missing values**를 먼저 제대로 이해하는 단계로 가는 게 좋아. 특히 `PoolQC`, `GarageType`, `Bsmt...` 같은 `NaN` 이 진짜 결측인지, 아니면 “해당 시설 없음”이라는 의미인지 확인해야 preprocessing 설계를 제대로 할 수 있어.

그다음에 correlation과 `OverallQual`, `GrLivArea` 같은 주요 feature 관계를 보면 흐름이 자연스럽다.

2026. 9. 6. 오후 10:00:04



77

2026. 9. 6. 오후 10:00:36



좋아, 다음은 **Missing Values Analysis** 가자. 이번 목표는 단순히 “NaN이 많다”가 아니라, 그 **NaN**이 진짜 결측인지 아니면 ‘해당 시설 없음’이라는 의미인지 구분하는 거야.

새 Markdown 셀:

```
Missing Value Analysis
```

Some features contain a large number of missing values.

Before deciding how to preprocess them, I examined whether the missing values represent unknown data or the absence of a property feature.

그다음 Code 셀:

```
missing_count = df.isnull().sum()
missing_percent = (missing_count / len(df)) * 100

missing_summary = pd.DataFrame({
 "MissingCount": missing_count,
 "MissingPercent": missing_percent
})

missing_summary = missing_summary[missing_summary["MissingCount"] > 0]
missing_summary = missing_summary.sort_values(
 by="MissingPercent",
 ascending=False
)

missing_summary.head(20)
```

이렇게 하면 아까보다 보기 좋게:

	MissingCount	MissingPercent
PoolQC	1453	99.52
MiscFeature	1406	96.30
...		

형태로 나올 거야.

그리고 바로 시각화도 하나 만들자:

```
top_missing = missing_summary.head(15)

plt.figure(figsize=(10, 6))
plt.barh(
 top_missing.index[::-1],
 top_missing["MissingPercent"][::-1]
)

plt.xlabel("Missing Values (%)")
plt.ylabel("Feature")
plt.title("Top 15 Features with Missing Values")
plt.show()
```

이건 교수님 영상에서도 설명하기 좋은 그래프야.

## 그런데 여기서 핵심

이제 실제 Kaggle 데이터 정의를 조금 확인해보면 다음 같은 feature들이 있어:

PoolQC	= Pool quality
Alley	= Type of alley access
Fence	= Fence quality
FireplaceQu	= Fireplace quality
GarageType	= Garage location
BsmtQual	= Basement quality

이런 컬럼에서 NaN 이면 상당수는:

```
PoolQC = NaN
→ pool information is missing
```

이 아니라

```
PoolQC = NaN
→ this house has no pool
```

일 가능성이 있어.

그래서 데이터를 삭제하거나 평균으로 채우기 전에 **feature** 의미를 반드시 확인해야 돼.

---

## 몇 개를 실제로 확인해보자

다음 셀:

```
columns_to_check = [
 "PoolQC",
 "Alley",
 "Fence",
 "FireplaceQu",
 "GarageType",
 "BsmtQual"
]

for col in columns_to_check:
 print(f"\n{col}")
 print(df[col].value_counts(dropna=False))
```

이걸 실행하면 예를 들어:

```
PoolQC
NaN 1453
Gd ...
Ex ...
```

같이 나올 거야.

여기서 중요한 건 당장 전부 외울 필요 없어.

우리가 확인하려는 질문은:

*이 feature의 NaN이 다른 property feature들과 논리적으로 연결되는가?*

예를 들어 PoolQC가 NaN인 집에서:

```
df.loc[df["PoolQC"].isna(), "PoolArea"].value_counts().head()
```

를 보면 대부분:

```
0
```

일 가능성이 높아.

이건 상당히 좋은 증거야.

*PoolQC = NaN인 이유는 PoolQC 데이터가 우연히 빠진 게 아니라  
**PoolArea = 0**, 즉 수영장이 없기 때문일 가능성이 높다.*

## Garage도 같은 방식으로 확인

```
df.loc[df["GarageType"].isna(), "GarageCars"].value_counts(dropna=False)
```

그리고 Basement:

```
df.loc[df["BsmtQual"].isna(), "TotalBsmtSF"].value_counts(dropna=False)
```

이 세 가지를 실행해보자:

```
df.loc[df["PoolQC"].isna(), "PoolArea"].value_counts().head()
```

```
df.loc[df["GarageType"].isna(), "GarageCars"].value_counts(dropna=False)
```

```
df.loc[df["BsmtQual"].isna(), "TotalBsmtSF"].value_counts(dropna=False)
```

## 왜 이걸 하나면

우리가 나중에 preprocessing에서:

```
SimpleImputer(strategy="most_frequent")
```

를 아무 생각 없이 쓰면,

```
GarageType = NaN
```

인 집을:

“가장 흔한 *GarageType*으로 채워버림”

이 될 수도 있어.

근데 그 집은 애초에 **garage가 없는 집**일 수 있잖아.

그럼 더 논리적인 처리는:

```
NaN → "None"
```

이 될 수 있어.

이게 바로 데이터 전처리에서 **domain meaning**을 이해해야 하는 이유야.

우선 위 3개 결과 보여줘.

그걸 보고 우리가 “**structural missing values**”와 “**actual missing values**”를 어떻게 나눌지 다음 단계에서 결정하자.

2026. 9. 6. 오후 10:00:41





```

[22] ✓ 0.0s Python
... PoolArea
0 1453
Name: count, dtype: int64

[23] ✓ 0.0s Python
... GarageCars
0 81
Name: count, dtype: int64

[24] ✓ 0.0s Python
... TotalBsmtSF
0 37
Name: count, dtype: int64

```

Generate + Code + Markdown

2026. 9. 6. 오후 10:03:43



## Thinking

- 해석 정리하기

현재 세 열의 결측값은 실제 누락보다 구조적 결측으로 보이고 있어요. `BsmtQual`은 지하실 없음, `PoolQC`는 수영장 없음, `GarageType`은 차고 없음일 가능성이 큼니다.

다음에는 연관된 수치형 열(`TotalBsmtSF`, `PoolArea`, `GarageCars`)이 정말 0인지 확인하고, 다른 결측 열들과 구분해 보겠습니다.

- 해석 정리 완료

✓ 10s 동안 처리함

오, 이 결과가 아주 깔끔하게 나왔어. 🍌

우리가 방금 세운 가설이 실제 데이터로 확인됐어.

결과를 하나씩 보면:

PoolQC is NaN  
→ PoolArea = 0

→ 1453 / 1453 houses

즉 `PoolQC` 가 비어 있는 1,453채는 전부 수영장 면적이 0이야.

`GarageType` is NaN

→ `GarageCars` = 0

→ 81 / 81 houses

마찬가지로 `garage type`이 비어 있는 81채는 전부 **garage capacity**가 0.

`BsmtQual` is NaN

→ `TotalBsmtSF` = 0

→ 37 / 37 houses

basement quality가 없는 37채도 전부 **basement** 면적이 0이야.

즉 적어도 이 세 변수에서는:

`NaN` = 데이터가 실수로 빠짐

이라기보다

`NaN` = 해당 시설이 없음

이라는 **structural missing value**로 보는 게 훨씬 타당해.

## Notebook에 이 Markdown을 넣자

```
Interpretation of Structural Missing Values
```

The missing-value analysis suggests that some missing values represent the absence of a property feature rather than unknown or corrupted data.

For example:

- All 1,453 houses with missing ``PoolQC`` have ``PoolArea` = 0``.
- All 81 houses with missing ``GarageType`` have ``GarageCars` = 0``.

- All 37 houses with missing `BsmtQual` have `TotalBsmtSF = 0`.

These results provide strong evidence that the missing values in these features are structural. In other words, the value is missing because the house does not have a pool, garage, or basement.

Therefore, these categorical missing values should not automatically be replaced with the most common category. A category such as `"None"` or `"NoGarage"` may preserve more meaningful information during preprocessing.

이건 나중에 YouTube에서도 설명하기 좋은 부분이야.

*"At first, I saw many missing values. Instead of immediately imputing them, I checked related variables and found that many NaNs actually represented the absence of a property feature."*

이 정도면 네가 왜 **preprocessing**을 그렇게 했는지 설명할 수 있지.

그런데 여기서 아주 재미있는 문제 하나를 발견했어.

아까 네 missing 결과에:

MasVnrType      872

가 있었지?

872개 missing은 상당히 많아. 그런데 이 데이터의 categorical 값 중에는 실제로 `"None"` 처럼 **"masonry veneer 없음"**을 나타내는 문자열이 들어갈 수 있어.

그리고 pandas의 `read_csv()` 는 일부 문자열을 자동으로 missing value로 해석할 수 있어.

즉 우리가 다음으로 확인해야 할 게:

**진짜 원본 데이터에서 비어 있었나? 아니면 pandas가 `"None"` / `"NA"` 같은 값을 `NaN`으로 변환했나?**

야.

이건 꽤 좋은 Data Science 포인트야. 😊

다음 셀에서 원본 CSV를 **자동 NA 변환 없이** 한 번만 다시 읽어보자:

```
raw_df = pd.read_csv("../data/train.csv", keep_default_na=False)

raw_df["MasVnrType"].value_counts().head(10)
```

그리고:

```
raw_df["Alley"].value_counts().head(10)
```

또:

```
raw_df["PoolQC"].value_counts().head(10)
```

이 세 개 결과를 보여줘.

우리가 지금 **“pandas가 무엇을 missing으로 바꿔버렸는지”**까지 확인하고 있는 거야. 여기서 꽤 중요한 preprocessing 결정이 나올 수 있어.

2026. 9. 6. 오후 10:03:52



```
raw_df = pd.read_csv("../data/train.csv", keep_default_na=False)

raw_df["MasVnrType"].value_counts().head(10)
```

[25] ✓ 0.0s Python

```
... MasVnrType
None 864
BrkFace 445
Stone 128
BrkCmn 15
NA 8
Name: count, dtype: int64
```

```
raw_df["Alley"].value_counts().head(10)
```

[26] ✓ 0.0s Python

```
... Alley
NA 1369
Grvl 50
Pave 41
Name: count, dtype: int64
```

```
raw_df["PoolQC"].value_counts().head(10)
```

[27] ✓ 0.0s Python

```
... PoolQC
NA 1453
Gd 3
Ex 2
Fa 2
Name: count, dtype: int64
```

2026. 9. 6. 오후 10:04:50



## Thinking

- 구분 정리하기

`keep\_default\_na=False`를 사용하면 실제 결측치와 `"NA"`, `None` 같은 도메인 코드를 구분할 수 있습니다. 특히 `NA`는 일부 열에서 기능 없음이라는 의미일 수 있어, 재분석 전에 각 열의 의미를 확인하고 결측치 처리를 조정하겠습니다.

- 구분을 정리함

✓ 9s 동안 처리함

오, 이거 **굉장히 중요한 걸 잡았어**. 지금 결과 때문에 처음의 `pd.read_csv()` 방식을 그대로 쓰면 안 된다는 게 드러났어.

핵심은 **CSV** 안의 `"NA"` 와 `None` 이 전부 같은 의미가 아니라는 것이야.

지금 결과를 보면:

- `PoolQC` : `"NA"` 1453개 → 여기서는 **No Pool**
- `Alley` : `"NA"` 1369개 → 여기서는 **No Alley Access**
- `MasVnrType` : `"None"` 864개 → **No Masonry Veneer**
- `MasVnrType` : `"NA"` 8개 → 이걸 오히려 실제 **missing/unknown**일 가능성이 큼

그런데 우리가 처음에 그냥:

```
df = pd.read_csv("../data/train.csv")
```

했을 때 pandas가 `"NA"` 나 `"None"` 같은 문자열을 자동으로 `NaN`으로 해석해버렸어.

그래서 아까:

```
MasVnrType 872 missing
```

라고 나온 게 사실은:

```
864 "None" ← 정상적인 범주: masonry veneer 없음
8 "NA" ← 실제 결측 가능성

872
```

가 합쳐져 버린 거야.

이건 **Data Science**에서 꽤 좋은 발견이야. 그냥 missing-value imputation으로 넘어갔으면 의미 있는 정보를 망가뜨릴 뻔했어.

Notebook에 이 Markdown을 남기자:

```
Important Finding: Domain Values vs. Missing Values
```

```
A second inspection of the raw CSV revealed an important data-loading issue.
```

```
Pandas automatically interprets strings such as `"NA"` and `"None"` as missing values under its default CSV parsing behavior. However, in this dataset, these strings can have domain-specific meanings.
```

```
Examples include:
```

- ``PoolQC = "NA"`` represents a house with no pool.
- ``Alley = "NA"`` represents no alley access.
- ``MasVnrType = "None"`` represents no masonry veneer.

Therefore, treating all of these values as generic missing data would remove meaningful information.

For example, ``MasVnrType`` contains 864 ``"None"`` observations and only 8 ``"NA"`` observations. The default Pandas import combined these into 872 missing values.

This means missing-value handling must consider the meaning of each feature rather than applying one global strategy.

그리고 여기서 중요한 결정이 하나 있어.

`keep_default_na=False` 인 `raw_df` 를 분석용 **main dataframe**으로 그냥 쓰는 것도 완벽한 해결책은 아니야.

왜냐하면 그러면 이번에는:

```
PoolQC "NA"
```

와

```
MasVnrType "NA"
```

를 똑같은 문자열로 보존하는데, 두 `"NA"` 의 의미가 서로 다를 수 있기 때문이야.

따라서 가장 좋은 방법은:

**원본 문자열을 보존해서 읽고 → *column* 의미에 따라 명시적으로 처리한다.**

이거야.

예를 들어 나중 preprocessing에서:

PoolQC:

"NA" → "NoPool"

Alley:

"NA" → "NoAlley"

GarageType:

"NA" → "NoGarage"

BsmtQual:

"NA" → "NoBasement"

MasVnrType:

"None" → "NoMasonryVeneer"

"NA" → 실제 missing으로 별도 처리

같은 식으로.

이게 그냥:

```
SimpleImputer(strategy="most_frequent")
```

하나 던지는 것보다 훨씬 논리적이지.

## 그런데 지금 당장 전처리하진 말자

아직 EDA 단계니까 `raw_df` 와 기존 `df` 둘 다 잠깐 두고, 어떤 **column**에서 **pandas** 자동 **NA** 처리가 영향을 주었는지 **정확히 확인**해보자.

다음 셀 하나만 실행해봐:

```
na_interpretation_effect = []

for col in raw_df.columns:
 default_missing = df[col].isna().sum()

 if raw_df[col].dtype == "object":
 literal_none = (raw_df[col] == "None").sum()
```



```

literal_na = (raw_df[col] == "NA").sum()

if default_missing > 0 or literal_none > 0 or literal_na > 0:
 na_interpretation_effect.append({
 "Feature": col,
 "PandasMissing": default_missing,
 "Literal_None": literal_none,
 "Literal_NA": literal_na
 })

na_effect_df = pd.DataFrame(na_interpretation_effect)

na_effect_df.sort_values(
 "PandasMissing",
 ascending=False
).head(30)

```

이 표가 꽤 중요할 거야.

이걸 보면:

어떤 **feature**에서 "NA" / "None" 을 **pandas**가 **NaN**으로 바꿨는가

를 한눈에 볼 수 있어.

결과 보여줘. 그러면 우리가 **House Prices**용 **missing-value** 처리 정책을 직접 설계해보자. 이 부분은 과제에서도 꽤 좋은 "AI-assisted analysis를 내가 검증했다" 사례가 될 수 있어.

2026. 9. 6. 오후 10:05:00



KeyError: 'PandasMissing'

2026. 9. 6. 오후 10:05:52



응, 이건 네 데이터 문제가 아니라 내 코드에서 문자열 컬럼을 `dtype == "object"` 로만 잡은 게 문제야.



아까 네 결과를 보면 문자열 타입이:

```
str 43
```

로 나왔잖아. 지금 pandas 버전에서는 문자열 컬럼이 예전처럼 `object` 가 아니라 `str` 타입으로 잡히고 있어서, 이 조건:

```
if raw_df[col].dtype == "object":
```

에 아무 컬럼도 걸리지 않은 거야.

그래서 `na_interpretation_effect` 가 빈 리스트가 되고:

```
pd.DataFrame(na_interpretation_effect)
```

도 빈 DataFrame이 된 다음, 존재하지 않는 `"PandasMissing"` 컬럼으로 정렬하려다가:

```
KeyError: 'PandasMissing'
```

가 난 거야.

**이 코드로 바꾸자**

```
from pandas.api.types import is_string_dtype
```

```
na_interpretation_effect = []
```

```

for col in raw_df.columns:
 if is_string_dtype(raw_df[col]):
 default_missing = df[col].isna().sum()
 literal_none = (raw_df[col] == "None").sum()
 literal_na = (raw_df[col] == "NA").sum()

 if default_missing > 0 or literal_none > 0 or literal_na > 0:
 na_interpretation_effect.append({
 "Feature": col,
 "PandasMissing": default_missing,
 "Literal_None": literal_none,
 "Literal_NA": literal_na
 })

na_effect_df = pd.DataFrame(na_interpretation_effect)

na_effect_df.sort_values(
 "PandasMissing",
 ascending=False
).head(30)

```

이제 정상적으로 표가 나올 거야.

왜 `is_string_dtype()` 가 더 좋은가

예전 pandas에서는 문자열이 보통:

```
object
```

였는데, 최근 버전에서는:

```
str
string
```

처럼 별도의 문자열 타입으로 처리되는 경우가 있어.

그래서:

```
dtype == "object"
```

보다:

```
is_string_dtype(...)
```

가 훨씬 안전해.

사실 이것도 notebook에 나름 재미있는 기록이야 ㅋㅋ. 라이브러리 버전에 따라 **datatype representation**이 달라질 수 있다는 실제 사례니까.

수정한 코드 실행해서 나오는 표 보여줘. 그걸 보고 정말 어떤 컬럼들이 pandas 자동 NA 처리의 영향을 받았는지 정리하자.

2026. 9. 6. 오후 10:05:57



	Feature	PandasMissing	Literal_None	Literal_NA
16	PoolQC	1453	0	1453
18	MiscFeature	1406	0	1406
1	Alley	1369	0	1369
17	Fence	1179	0	1179
2	MasVnrType	872	864	8
10	FireplaceQu	690	0	690
0	LotFrontage	259	0	259
11	GarageType	81	0	81
12	GarageYrBlt	81	0	81
13	GarageFinish	81	0	81
14	GarageQual	81	0	81
15	GarageCond	81	0	81
8	BsmtFinType2	38	0	38
6	BsmtExposure	38	0	38
7	BsmtFinType1	37	0	37
5	BsmtCond	37	0	37
4	BsmtQual	37	0	37
3	MasVnrArea	8	0	8
9	Electrical	1	0	1

2026. 9. 6. 오후 10:06:38



## Thinking

- 정제 정책 설정하기

구조적 부재와 실제 결측을 구분하는 전처리 방침을 세우고 있어요. 공식 데이터 설명을 확인해 범주형 코드는 의미 있는 부재값으로 바꾸고, 수치형 결측은 별도로 처리하려고 합니다.

- 정제 정책 설정 완료

✓ 39s 동안 처리함

오 이 표는 꽤 중요한 결과야. 우리가 일부러 확인해보길 잘했어. ☺

핵심은 `PandasMissing` 이 사실 두 종류를 섞어서 세고 있었다는 거야. 특히 `MasVnrType` 이 결정적인 증거야.



Feature	실제 의미
PoolQC : NA 1453	대부분 <b>No Pool</b>
Alley : NA 1369	<b>No Alley Access</b>
Fence : NA 1179	<b>No Fence</b>
FireplaceQu : NA 690	<b>No Fireplace</b>
GarageType : NA 81	앞에서 확인했듯 <b>No Garage</b>
BsmtQual : NA 37	앞에서 확인했듯 <b>No Basement</b>
MasVnrType : None 864	<b>No Masonry Veneer</b> 라는 정상 범주
MasVnrType : NA 8	실제 missing 가능성이 높은 별도 8개
LotFrontage : NA 259	실제 숫자값이 없는 경우 → imputation 대상
Electrical : NA 1	실제 categorical missing 가능성

특히:

```
MasVnrType
PandasMissing = 872
```

하지만 실제로는:

```
None = 864
NA = 8

 872
```

였던 거지.

즉 그냥:

```
pd.read_csv("../data/train.csv")
```

하면 Pandas가 서로 다른 의미를 하나의 NaN으로 합쳐버린다.

이거 자체가 이번 과제에서 꽤 좋은 Data Science finding이야.

---

## Notebook에 먼저 기록

Markdown 셀 하나 추가하자.

```
Missing-Value Parsing Finding
```

The raw CSV inspection revealed that Pandas' default missing-value parsing can remove important domain information.

In particular, the default import treated both `"NA"` and `"None"` as missing values.

For example, `MasVnrType` appeared to contain 872 missing values under the default Pandas import. However, the raw CSV actually contains:

- 864 `"None"` values, representing houses with no masonry veneer
- 8 `"NA"` values, representing genuinely unavailable values

Similar `"NA"` values in features such as `PoolQC`, `GarageType`, `BsmtQual`, and `FireplaceQu` represent the absence of a property feature rather than corrupted data.

Therefore, the dataset will be loaded with the original string values preserved, and missing values will be handled according to the meaning of each feature.

이 정도면 교수님이 봐도:

“*시*가 그냥 *imputer* 던진 게 아니라 실제 데이터를 확인했네.”

가 보여.

---

## 이제 main dataframe을 제대로 만들자

앞으로는 기존 `df` 보다 `raw_df` 를 기준으로 **\*\*깨끗한 `clean_df`**를 만들자.

새 Markdown:

```
Domain-Aware Data Cleaning
```

Based on the missing-value investigation, I created a cleaned copy of the dataset that preserves meaningful absence categories while keeping genuine unknown values as missing data.

그리고 코드:

```
clean_df = raw_df.copy()
```

먼저 명백하게 **시설 없음**을 의미하는 categorical 값들:

```
absence_mapping = {
 "PoolQC": "NoPool",
 "Alley": "NoAlley",
 "Fence": "NoFence",
 "MiscFeature": "NoMiscFeature",
 "FireplaceQu": "NoFireplace"
}

for col, label in absence_mapping.items():
 clean_df[col] = clean_df[col].replace("NA", label)
```

Garage:

```
garage_columns = [
 "GarageType",
 "GarageFinish",
 "GarageQual",
 "GarageCond"
]

for col in garage_columns:
 clean_df.loc[
```



```

 (clean_df["GarageCars"] == 0) & (clean_df[col] == "NA"),
 col
] = "NoGarage"

```

Basement는 조금 더 신중하게 한다.

왜냐하면:

```

BsmtQual 37
BsmtExposure 38

```

이었잖아?

즉 basement 없는 집 37채 말고 **추가 missing** 하나가 존재할 가능성이 있어.

그래서 무조건 `NA → NoBasement` 하지 말고:

```

basement_columns = [
 "BsmtQual",
 "BsmtCond",
 "BsmtExposure",
 "BsmtFinType1",
 "BsmtFinType2"
]

for col in basement_columns:
 clean_df.loc[
 (clean_df["TotalBsmtSF"] == 0) & (clean_df[col] == "NA"),
 col
] = "NoBasement"

```

이게 꽤 중요해.

`TotalBsmtSF == 0`

인 경우에만:

`"NA"` = *No Basement*

라고 결론 내리는 거야.

그런데 basement가 실제로 있는데 "NA" 라면 진짜 **missing**으로 남겨야 하니까:

```
for col in basement_columns:
 clean_df[col] = clean_df[col].replace("NA", np.nan)
```

## Masonry Veneer도 따로 처리

아까 발견한 핵심:

```
clean_df["MasVnrType"] = clean_df["MasVnrType"].replace({
 "None": "NoMasonryVeneer",
 "NA": np.nan
})
```

그러면:

```
864개 → NoMasonryVeneer
8개 → NaN
```

으로 의미가 보존돼.

## 숫자형 missing 다시 숫자로 변환

`keep_default_na=False` 로 읽었기 때문에 NA 가 들어있던 숫자 column은 문자열이 되어 있을 수 있어.

그래서:

```
numeric_missing_columns = [
 "LotFrontage",
 "GarageYrBlt",
 "MasVnrArea"
]

for col in numeric_missing_columns:
 clean_df[col] = pd.to_numeric(
 clean_df[col].replace("NA", np.nan),
```

```
errors="coerce"
)
```

그리고 `Electrical` 의 1개:

```
clean_df["Electrical"] = clean_df["Electrical"].replace("NA", np.nan)
```

## 이제 결과 확인

한 셀:

```
remaining_missing = clean_df.isnull().sum().sort_values(ascending=False)

remaining_missing[remaining_missing > 0]
```

그리고:

```
clean_df.dtypes.value_counts()
```

이 두 개 실행해봐.

이제 처음처럼:

```
PoolQC 1453
Alley 1369
...
```

가 쏟아지는 게 아니라 정말 처리가 필요한 **missing values**만 상당히 줄어서 남아야 해.

여기서부터 우리는 그 진짜 missing들에 대해서만:

*numeric → median?*

*categorical → most frequent / Unknown?*

같은 **imputation 전략**을 고민하면 돼.

그리고 지금 한 과정이 꽤 중요한 개념이야:

## Missing-value handling ≠ 그냥 NaN 채우기

오히려:

“왜 이 값이 비어 있는가?” → 의미 확인 → 그 다음 처리

가 제대로 된 Data Science야.

2026. 9. 6. 오후 10:06:55



```
clean_df.dtypes.value_counts()
```

✓ 0.0s

```
str 43
int64 35
float64 3
Name: count, dtype: int64
```

와 근데 따라는 가고 있는데, 뭐가 어떻게 돌아가고 있는지 모르겠다  
허허허

2026. 9. 6. 오후 10:09:50



ㅋㅋ 지금 딱 멈추는 게 맞아. 코드가 너무 빨리 복잡해졌어. **지금까지 한 걸 머릿속 그림으로 먼저 잡고 가자.** 사실 우리가 한 건 생각보다 단순해.

처음 파일을 읽었을 때는 이렇게 했지:

```
df = pd.read_csv("../data/train.csv")
```

문제는 Pandas가 CSV 안의 "NA"와 "None"을 보고 자동으로 **\*\*\*"아, 빈 값이구나"\*\*\***라고 판단해버렸다는 거야.

그런데 House Prices 데이터에서는 "NA"가 항상 "데이터 없음"이 아니었어.

원본 CSV

PoolQC = "NA" → 실제 의미: 수영장 없음  
GarageType = "NA" → 실제 의미: 차고 없음  
BsmtQual = "NA" → 실제 의미: 지하실 없음

MasVnrType = "None" → 실제 의미: masonry veneer 없음  
MasVnrType = "NA" → 이건 진짜 정보가 없는 경우

그런데 Pandas 기본 설정은 이것들을 몽땅:

```
"NA" / "None"
 ↓
NaN
```

으로 만들어버린 거야.

그래서 우리가 두 번째로:

```
raw_df = pd.read_csv(
 "../data/train.csv",
 keep_default_na=False
)
```

라고 했어.

뜻은 그냥:

**“Pandas야, 네가 마음대로 NA를 빈 값으로 바꾸지 말고 CSV에 적힌 그대로 읽어.”**

야.

그래서 현재 데이터가 세 단계로 존재한다고 생각하면 돼.

```
train.csv
 |
 └─ df
 | Pandas 기본 방식으로 읽음
 | "NA", "None" → NaN으로 바뀜
 |
 └─ raw_df
 원본 문자열 그대로 읽음
 "NA", "None" 그대로 보존
 |
 ↓
 clean_df
```

우리가 의미를 확인하면서  
올바르게 정리한 데이터

`clean_df`에서는 정확히 뭘 한 거냐

예를 들어 원래:

```
PoolQC = "NA"
```

였는데 다른 컬럼을 확인해보니까:

```
PoolArea = 0
```

이었잖아.

그래서 우리가 판단했어.

*“아, PoolQC를 모르는 게 아니라 수영장이 없는 집이구나.”*

그래서:

```
"NA"
↓
"NoPool"
```

로 바꿨어.

Garage도:

```
GarageType = "NA"
GarageCars = 0

→ "NoGarage"
```

Basement도:

```
BsmtQual = "NA"
TotalBsmtSF = 0
```

→ "NoBasement"

이렇게 한 거야.

반대로 진짜 값이 없는 것은 NaN으로 남겨두고, 나중에 ML 직전에 **imputation**하려는 거고.

그러면 지금 네 화면의 이건 무슨 뜻?

```
str 43
int64 35
float64 3
```

이건 “청소가 몇 % 됐다”는 뜻이 아니야.

그냥 81개 column의 데이터 종류를 세어본 것이야.

```
43 + 35 + 3 = 81 columns
```

즉:

```
str 43
→ Neighborhood, GarageType처럼 문자/카테고리 데이터

int64 35
→ OverallQual, BedroomAbvGr처럼 정수 데이터

float64 3
→ LotFrontage처럼 소수점/NaN이 포함될 수 있는 숫자 데이터
```

예를 들어:

```
OverallQual
```

이런 건:

```
1, 2, 3, ... 10
```



→ 숫자니까 `int64`

반면:

```
Neighborhood
```

은:

```
CollgCr
```

```
Veenker
```

```
Crawfor
```

```
...
```

→ 문자니까 `str`.

그래서 나중에 ML 모델에게 넣을 때도 처리법이 달라져.

```
clean_df
```

```
|
```

```
|— 숫자 38개
```

```
|
```

```
↓
```

```
|
```

```
missing 보완
```

```
|
```

```
|— 문자 43개
```

```
↓
```

```
missing 보완
```

```
↓
```

```
One-Hot Encoding
```

```
↓
```

```
ML Model
```

ML 모델은 기본적으로:

```
"NoGarage"
```

```
"CollgCr"
```

같은 글자를 그대로 이해하지 못하니까 나중에 숫자로 바꾸는 과정이 필요한 거야.

## 지금까지 한 걸 한 문장씩만 기억하면

1. 데이터를 읽었다.
2. 어떤 데이터가 들어있는지 살펴봤다.
3. **SalePrice**가 한쪽으로 치우쳐 있다는 걸 발견했다.
4. **missing value**가 많다는 걸 발견했다.
5. 그런데 **missing**처럼 보이는 것 중 일부는 사실 “시설 없음”이라는 정보였다.
6. 그래서 원본 의미를 보존하도록 데이터를 다시 정리했다.

아직 **Machine Learning**은 하나도 안 했어.

지금은 요리로 치면:

재료 꺼냄 → 상했는지 확인 → 씻음 → 손질

까지만 한 거야.

모델링은 아직 불도 안 켜어 ㅋㅋ.

그리고 내가 보기엔 이 부분을 notebook에 넣은 건 오히려 잘했어. 흔한 과제라면 그냥:

```
SimpleImputer(...)
```

쳐버리고 넘어가는데, 우리는 왜 값이 없는지 확인하고 전처리를 결정했기 때문이야. 교수님이 원하는 “data understanding”에 더 가까워.

다만 여기서 더 깊게 **missing-value** 코드를 파고들지는 말자. 지금 수준이면 충분해. 다음으로 넘어가기 전에 `clean_df`에 진짜 **missing**이 뭐가 남았는지 한 번 확인하고, 그다음 훨씬 직관적인 **Correlation / OverallQual vs SalePrice** 같은 그래프로 넘어가자. 그쪽부터는 훨씬 그림이 잘 그려질 거야.

2026. 9. 6. 오후 10:10:00



허허 이해는 나중에 하고 일단 속제 완료부터하자

2026. 9. 6. 오후 10:20:30



좋아 ㅋㅋ 그럼 이해는 나중에 복습용으로 따로 정리하고, 지금은 제출 가능한 완성본 만드는 모드로 가자.

이제부터는 내가 최소 설명 + 실행할 코드 + 결과 해석 문장만 줄게.

흐름은:

**EDA → preprocessing → baseline → 3 models → comparison → interpretation → README → GitHub → video**

로 끝까지 밀자.

다음 단계는 바로 **Correlation + 주요 feature** 시각화야. Notebook에 아래 셀 추가해:

```
numeric_df = clean_df.select_dtypes(include=["int64", "float64"])

correlations = (
 numeric_df.corr(numeric_only=True)["SalePrice"]
 .sort_values(ascending=False)
)

correlations.head(15)
```

그 다음 그래프:

```
top_features = correlations.drop("SalePrice").head(10)

plt.figure(figsize=(10, 6))
plt.barh(top_features.index[::-1], top_features.values[::-1])
plt.xlabel("Correlation with SalePrice")
plt.title("Top Numerical Features Correlated with SalePrice")
plt.show()
```

그리고 주요 관계 2개만:



```
plt.figure(figsize=(8, 5))
plt.scatter(clean_df["OverallQual"], clean_df["SalePrice"], alpha=0.5)
plt.xlabel("OverallQual")
plt.ylabel("SalePrice")
plt.title("Overall Quality vs SalePrice")
plt.show()
```

```
plt.figure(figsize=(8, 5))
plt.scatter(clean_df["GrLivArea"], clean_df["SalePrice"], alpha=0.5)
plt.xlabel("Above Ground Living Area")
plt.ylabel("SalePrice")
plt.title("GrLivArea vs SalePrice")
plt.show()
```

이 세 개 실행하고 **correlation** 상위 10개 결과만 붙여줘.

그 다음부터 바로 preprocessing pipeline 만들고 모델 돌리자.

이번엔 진짜 속제 완성 우선으로 쪽 간다.

2026. 9. 6. 오후 10:20:36



```
SalePrice 1.000000 OverallQual 0.790982
GrLivArea 0.708624 GarageCars 0.640409
GarageArea 0.623431 TotalBsmtSF 0.613581
1stFlrSF 0.605852 FullBath 0.560664
TotRmsAbvGrd 0.533723 YearBuilt 0.522897
YearRemodAdd 0.507101 GarageYrBlt 0.486362
MasVnrArea 0.477493 Fireplaces 0.466929
BsmtFinSF1 0.386420
```

2026. 9. 6. 오후 10:21:16



좋아. 결과도 정상적이야. 핵심은 `OverallQual (0.79)` 와 `GrLivArea (0.71)` 가 가격과 가장 강한 양의 관계를 보인다는 정도만 기록하고 바로 **전처리 + 모델링**으로 넘어가자.

Notebook에 짧게 Markdown 하나:

### ### Correlation Findings

Among the numerical features, ``OverallQual`` shows the strongest correlation with ``SalePrice`` (approximately 0.79), followed by ``GrLivArea`` (approximately 0.71).

Garage capacity, garage area, basement area, and first-floor area also show moderately strong positive relationships with house prices.

These relationships suggest that overall property quality and usable living space are important predictors of sale price.

이제 **Data Preparation** 시작.

## 1. 필요한 sklearn import

새 코드 셀:

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.dummy import DummyRegressor
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor

from sklearn.metrics import (
 root_mean_squared_error,
 mean_absolute_error,
 r2_score
)
```

---

## 2. X / y 나누기

```
X = clean_df.drop(columns=["SalePrice"])
y = clean_df["SalePrice"]

X_train, X_valid, y_train, y_valid = train_test_split(
 X,
 y,
 test_size=0.2,
 random_state=42
)

print("Training set:", X_train.shape)
print("Validation set:", X_valid.shape)
```

대략:

```
Training set: (1168, 80)
Validation set: (292, 80)
```

가 나와야 해.

### 3. 숫자/문자 컬럼 분리

```
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns
categorical_features = X.select_dtypes(include=["str", "object"]).columns

print("Numeric features:", len(numeric_features))
print("Categorical features:", len(categorical_features))
```

### 4. 전처리 Pipeline

```
numeric_pipeline = Pipeline([
 ("imputer", SimpleImputer(strategy="median")),
 ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
 ("imputer", SimpleImputer(strategy="most_frequent")),
 ("encoder", OneHotEncoder(
 handle_unknown="ignore",
 sparse_output=False
))
])

preprocessor = ColumnTransformer([
 ("num", numeric_pipeline, numeric_features),
 ("cat", categorical_pipeline, categorical_features)
])
```

여기까지가:

숫자  
→ missing은 median  
→ scaling

문자

- 진짜 missing만 most frequent
- One-Hot Encoding

이야.

---

## 5. 이제 모델 4개 한번에 돌리자

```
models = {
 "Dummy Baseline": DummyRegressor(strategy="mean"),
 "Ridge Regression": Ridge(alpha=1.0),
 "Random Forest": RandomForestRegressor(
 n_estimators=300,
 random_state=42,
 n_jobs=-1
),
 "Gradient Boosting": GradientBoostingRegressor(
 random_state=42
)
}
```

그리고:

```
results = []

trained_models = {}

for name, model in models.items():

 pipeline = Pipeline([
 ("preprocessor", preprocessor),
 ("model", model)
])

 pipeline.fit(X_train, y_train)

 predictions = pipeline.predict(X_valid)
```



```
rmse = root_mean_squared_error(y_valid, predictions)
mae = mean_absolute_error(y_valid, predictions)
r2 = r2_score(y_valid, predictions)
```

```
results.append({
 "Model": name,
 "RMSE": rmse,
 "MAE": mae,
 "R2": r2
})
```

```
trained_models[name] = pipeline
```

```
results_df = pd.DataFrame(results).sort_values("RMSE")
```

```
results_df
```

🔥 여기가 드디어 첫 **Machine Learning** 결과야.

표가:

Model	RMSE	MAE	R <sup>2</sup>
...	...	...	...

형태로 나올 거야.

그 결과를 그대로 보내줘.

그러면 바로 **1등 모델 선택 → cross-validation → predicted vs actual → feature importance → error analysis**까지 밀어서 Part 1 분석을 끝내자.

2026. 9. 6. 오후 10:21:30



	Model	RMSE	MAE	R2
3	Gradient Boosting	27790.020491	17164.860489	0.899315
2	Random Forest	29023.695652	17494.975502	0.890177
1	Ridge Regression	29720.634331	19101.405737	0.884840
0	Dummy Baseline	87619.034506	62575.926452	-0.000882

2026. 9. 6. 오후 10:25:56



좋아. 이걸 결과가 아주 깔끔해. **Gradient Boosting**이 현재 **validation** 기준 **1등**이야.

핵심만 보면:

- Gradient Boosting: RMSE  $\approx$  **27,790**, MAE  $\approx$  **17,165**,  $R^2 \approx$  **0.899**
- Random Forest: RMSE  $\approx$  **29,024**
- Ridge: RMSE  $\approx$  **29,721**
- Dummy baseline: RMSE  $\approx$  **87,619**

즉 Gradient Boosting이 단순 평균 예측 baseline보다 훨씬 낮고, validation data의 가격 변동을 약 **89.9% 설명**하고 있어.

Notebook에 이 Markdown 넣자:

```
Initial Model Comparison
```

```
The Gradient Boosting model achieved the best validation performance among the tested models.
```

- RMSE: approximately \$27,790
- MAE: approximately \$17,165
- $R^2$ : approximately 0.899

```
Random Forest and Ridge Regression also performed substantially better than the Dummy baseline.
```

```
The Dummy baseline produced an RMSE of approximately $87,619, showing that
```

the machine learning models provide a large improvement over simply predicting the average house price.

Based on the initial validation results, Gradient Boosting is the leading candidate for the final model.

이제 바로 **cross-validation**로 한 번 더 검증하자. 단일 train/validation split에서 우연히 잘 나온 결과도 있으니까.

```
from sklearn.model_selection import cross_val_score

best_pipeline = trained_models["Gradient Boosting"]

cv_scores = cross_val_score(
 best_pipeline,
 X,
 y,
 cv=5,
 scoring="neg_root_mean_squared_error",
 n_jobs=-1
)

cv_rmse = -cv_scores

print("Fold RMSE:", cv_rmse)
print("Mean CV RMSE:", cv_rmse.mean())
print("Std CV RMSE:", cv_rmse.std())
```

그 결과 보여줘.

그 다음은 딱 세 개만 하면 Part 1 모델 분석 거의 끝나:

**Predicted vs Actual → 큰 prediction error 확인 → feature importance**

여기까지 가면 README에 넣을 최종 결과도 거의 확정할 수 있어.

2026. 9. 6. 오후 10:25:58



```
Fold RMSE: [23534.66421654 32806.16651018
27905.73560421 22541.65014612 27567.98442384]
Mean CV RMSE: 26871.240180176137 Std CV RMSE:
3651.220970855307
```

2026. 9. 6. 오후 10:28:21



좋아. 이 정도면 **Gradient Boosting**이 우연히 한 **split**에서만 잘 나온 건 아니라는 근거가 생겼어.

5-fold RMSE가:

```
23,535
32,806
27,906
22,542
27,568
```

이고,

```
Mean CV RMSE ≈ $26,871
Std ≈ $3,651
```

이니까 평균적으로도 initial validation RMSE \$27,790 과 비슷한 수준이야. Fold마다 어느 정도 변동은 있지만 전반적으로 일관된 편이야.

Notebook에 이 Markdown 넣자:

```
Cross-Validation Results
```

```
To verify that the Gradient Boosting result was not caused by a favorable
single train-validation split, I evaluated the model using 5-fold
cross-validation.
```

```
The mean cross-validation RMSE was approximately $26,871, with a standard
deviation of approximately $3,651.
```

This result is similar to the original validation RMSE of approximately \$27,790, providing additional evidence that the model generalizes reasonably well across different subsets of the training data.

Although performance varies between folds, Gradient Boosting remains the strongest candidate for the final model.

이제 Part 1 분석은 세 가지만 더 하자.

## 1. Predicted vs Actual

```
final_model = trained_models["Gradient Boosting"]

final_predictions = final_model.predict(X_valid)

plt.figure(figsize=(7, 7))

plt.scatter(
 y_valid,
 final_predictions,
 alpha=0.6
)

min_price = min(y_valid.min(), final_predictions.min())
max_price = max(y_valid.max(), final_predictions.max())

plt.plot(
 [min_price, max_price],
 [min_price, max_price],
 linestyle="--"
)

plt.xlabel("Actual SalePrice")
plt.ylabel("Predicted SalePrice")
plt.title("Actual vs Predicted House Prices")

plt.show()
```

점이 대각선에 가까울수록 prediction이 좋은 거야.

## 2. 가장 크게 틀린 집 확인

```
error_df = pd.DataFrame({
 "Actual": y_valid,
 "Predicted": final_predictions
})

error_df["Error"] = error_df["Predicted"] - error_df["Actual"]
error_df["AbsoluteError"] = error_df["Error"].abs()

error_df.sort_values(
 "AbsoluteError",
 ascending=False
).head(10)
```

여기서는 모델이 어떤 집에서 크게 틀리는지 본다.

## 3. Feature Importance

Gradient Boosting은 어떤 feature를 많이 사용했는지도 볼 수 있어.

```
feature_names = final_model.named_steps[
 "preprocessor"
].get_feature_names_out()

importance_values = final_model.named_steps[
 "model"
].feature_importances_

feature_importance_df = pd.DataFrame({
 "Feature": feature_names,
 "Importance": importance_values
}).sort_values(
 "Importance",
 ascending=False
)
```

```
feature_importance_df.head(15)
```

그리고 그래프:

```
top_importance = feature_importance_df.head(15)

plt.figure(figsize=(10, 7))

plt.barh(
 top_importance["Feature"][:-1],
 top_importance["Importance"][:-1]
)

plt.xlabel("Feature Importance")
plt.title("Top 15 Gradient Boosting Feature Importances")

plt.show()
```

이 세 개 실행하고 `error_df` 상위 10개랑 **feature importance** 상위 15개 결과만 보여줘.

그러면 그걸 바탕으로 최종 interpretation 작성하고, **Part 1 notebook** 분석 자체는 사실상 끝내자.  
그 다음은 결과 저장 → README → GitHub로 넘어가면 돼.

2026. 9. 6. 오후 10:28:26



	Actual	Predicted	Error	AbsoluteError
898	611657	403266.454402	-208390.545598	208390.545598
691	755000	623130.716719	-131869.283281	131869.283281
581	253293	374979.540508	121686.540508	121686.540508
1046	556581	437505.100851	-119075.899149	119075.899149
774	395000	295771.766238	-99228.233762	99228.233762
261	276000	366728.240186	90728.240186	90728.240186
218	311500	228829.839055	-82670.160945	82670.160945
588	143000	211655.338051	68655.338051	68655.338051
58	438780	371406.377276	-67373.622724	67373.622724
1243	465000	399233.076708	-65766.923292	65766.923292

2026. 9. 6. 오후 10:29:51



좋아, 이 결과도 충분히 쓸 만해. 여기서 보이는 핵심은 큰 오차가 주로 비싼 집들에서 발생한다는 점이야.

예를 들면:

- 실제 \$611,657 → 예측 약 \$403,266 → 약 **\$208k** 과소예측
- 실제 \$755,000 → 예측 약 \$623,131 → 약 **\$132k** 과소예측
- 실제 \$556,581 → 예측 약 \$437,505 → 약 **\$119k** 과소예측

반대로 실제 \$253,293짜리를 약 \$374,980으로 예측한 것처럼 큰 **과대예측**도 있어.

그래서 모델 전체 성능은 좋지만, **극단적인 가격대나 특이한 집에서는 오차가 커질 수 있다**고 해석하면 돼.

Notebook에 이 Markdown 넣자:

```
Prediction Error Analysis
```

```
The Gradient Boosting model performs well overall, but the largest errors show that some individual properties are difficult to predict accurately.
```

```
Several of the largest errors occur for high-priced homes. For example, a house sold for approximately $611,657 was predicted at about $403,266, resulting in an underprediction of more than $200,000.
```



The model also produced some large overpredictions for mid-priced homes.

These results suggest that unusual or expensive properties may have characteristics that are not fully captured by the model. This is also consistent with the right-skewed `SalePrice` distribution observed earlier.

Therefore, although the overall RMSE and  $R^2$  are strong, individual prediction errors can still be substantial, especially for less typical properties.

이 정도면 충분해. 굳이 개별 row를 더 깊게 파헤치지는 말자.

이제 **Feature Importance** 결과만 확인하면 분석 파트 거의 끝이야.

아까 코드:

```
feature_importance_df.head(15)
```

실행해서 나오는 상위 15개 feature 보여줘.

그거 받으면 내가 바로:

1. 최종 feature interpretation
2. Part 1 conclusion
3. 결과 CSV/그래프 저장 코드
4. notebook final cleanup
5. README 업데이트

순서로 꼭 마무리해줄게.

2026. 9. 6. 오후 10:29:56



	Feature	Importance
4	num__OverallQual	0.501739
16	num__GrLivArea	0.153685
26	num__GarageCars	0.042024
12	num__TotalBsmtSF	0.035139
9	num__BsmtFinSF1	0.033492
13	num__1stFlrSF	0.029451
14	num__2ndFlrSF	0.027656
3	num__LotArea	0.018394
6	num__YearBuilt	0.017775
183	cat__BsmtQual_Ex	0.015935
19	num__FullBath	0.011334
24	num__Fireplaces	0.011290
7	num__YearRemodAdd	0.010584
229	cat__KitchenQual_Ex	0.007611
247	cat__GarageType_Attchd	0.006607

2026. 9. 6. 오후 10:31:19



좋아, 이걸로 **Part 1** 분석의 핵심 결과는 사실상 다 나왔어. 이제 정리하고 저장만 하면 돼.

Feature importance는 아주 명확해:

- OverallQual  $\approx$  **50.2%**
- GrLivArea  $\approx$  **15.4%**
- GarageCars  $\approx$  **4.2%**
- TotalBsmtSF  $\approx$  **3.5%**
- BsmtFinSF1  $\approx$  **3.3%**

즉 이 Gradient Boosting 모델에서는 **집의 전반적인 품질과 실제 거주 공간 크기**가 가격 예측에 가장 크게 기여했어.

Notebook에 이 Markdown 넣자:

### ### Feature Importance Interpretation

The Gradient Boosting model indicates that `OverallQual` is the most important predictive feature by a large margin, accounting for approximately 50% of the model's total feature importance.

`GrLivArea` is the second most important feature, followed by garage capacity, basement area, finished basement area, and first-floor area.

This result is consistent with the earlier correlation analysis, where `OverallQual` and `GrLivArea` also showed the strongest relationships with `SalePrice`.

The model therefore suggests that the overall quality of a house and the amount of usable living space are two of the strongest predictors of sale price in this dataset.

Feature importance reflects how the model uses the variables for prediction and should not be interpreted as proving a causal relationship.

마지막 문장은 넣는 게 좋아.

**feature importance = 원인은 아니니까.**

그리고 이제 최종 Conclusion도 바로 넣자:

### ## Final Conclusion

This project used an AI-assisted, end-to-end data science workflow to predict residential house prices using the Kaggle House Prices dataset.

The analysis included data understanding, missing-value investigation, exploratory data analysis, preprocessing, model comparison, cross-validation, prediction-error analysis, and model interpretation.

Four regression approaches were evaluated:

- Dummy Regressor
- Ridge Regression

- Random Forest Regressor
- Gradient Boosting Regressor

Gradient Boosting achieved the strongest validation performance:

- RMSE: approximately \$27,790
- MAE: approximately \$17,165
- R<sup>2</sup>: approximately 0.899

The 5-fold cross-validation mean RMSE was approximately \$26,871, providing additional evidence that the model performs consistently across different subsets of the data.

The most important predictive features were `OverallQual` and `GrLivArea`, suggesting that overall property quality and living space play major roles in house-price prediction.

The error analysis also showed that some unusual and high-priced properties remain difficult to predict accurately. Therefore, strong overall metrics do not guarantee accurate predictions for every individual property.

AI tools assisted with planning, coding, debugging, and interpretation, but the outputs were executed, inspected, and interpreted before being included in the final project.

이제 결과 파일도 실제로 저장하자.

## 모델 비교 결과 저장

```
results_df.to_csv(
 "../results/model_comparison.csv",
 index=False
)
```

## Cross-validation 결과 저장

```

cv_results_df = pd.DataFrame({
 "Fold": range(1, len(cv_rmse) + 1),
 "RMSE": cv_rmse
})

cv_results_df.to_csv(
 "../results/cross_validation_rmse.csv",
 index=False
)

```

## Prediction errors 저장

```

error_df.sort_values(
 "AbsoluteError",
 ascending=False
).to_csv(
 "../results/prediction_errors.csv",
 index=True
)

```

## Feature importance 저장

```

feature_importance_df.to_csv(
 "../results/feature_importance.csv",
 index=False
)

```

그리고 그래프도 GitHub에 파일로 남기는 게 좋아. 지금 그래프를 다시 생성할 때 `plt.show()` 직전에:

```

plt.savefig(
 "../figures/feature_importance.png",
 dpi=150,
 bbox_inches="tight"
)

```

처럼 넣으면 돼.

최종적으로 최소 이 네 개는 저장하자:

```
figures/
├─ saleprice_distribution.png
├─ top_correlations.png
├─ actual_vs_predicted.png
└─ feature_importance.png
```

그리고 `results/`:

```
results/
├─ model_comparison.csv
├─ cross_validation_rmse.csv
├─ prediction_errors.csv
└─ feature_importance.csv
```

이것까지 하면 **분석 내용은 완료**라고 봐도 돼.

그 다음 중요한 게 하나 있어.

Notebook 위에서 우리가 이것저것 테스트하면서 실패한 셀도 있었잖아. `KeyError` 같은 것들. 최종 제출에는 그런 시행착오 셀을 굳이 남길 필요 없어.

그래서 다음 단계는:

**Notebook 정리 → Restart Kernel → Run All → 결과 재검증 → 저장**

이걸 하자.

그거 끝나면 바로 **Part 1 README**를 최종 결과 기준으로 작성하고 GitHub에 첫 commit 올리자.

2026. 9. 6. 오후 10:31:23



```
...
```

```
figures/
```

```
...
```

여기에 지금 feature\\_importance.png 밖에 없어..

2026. 9. 6. 오후 10:42:24



응 ㅋㅋ 정상(?)이야. 우리가 앞에서 그래프들은 **화면에만** `plt.show()` 했고 파일로 저장은 안 했기 때문이야.

`feature_importance.png` 만 마지막에 `plt.savefig()` 를 넣어서 저장된 거고.

지금은 앞의 그래프 셀들을 다시 실행하면서 `plt.show()` 바로 전에 `plt.savefig()` 한 줄씩 추가하면 돼.

## 1. SalePrice Distribution

기존 셀을 이렇게 바꿔:

```
plt.figure(figsize=(8, 5))

plt.hist(df["SalePrice"], bins=30, edgecolor="black")

plt.xlabel("Sale Price")
plt.ylabel("Number of Houses")
plt.title("Distribution of SalePrice")

plt.savefig(
 "../figures/saleprice_distribution.png",
 dpi=150,
 bbox_inches="tight"
)

plt.show()
```

---

## 2. Top Correlations

```
top_features = correlations.drop("SalePrice").head(10)

plt.figure(figsize=(10, 6))

plt.barh(
 top_features.index[::-1],
 top_features.values[::-1]
)

plt.xlabel("Correlation with SalePrice")
plt.title("Top Numerical Features Correlated with SalePrice")

plt.savefig(
 "../figures/top_correlations.png",
 dpi=150,
 bbox_inches="tight"
)

plt.show()
```

---

## 3. Actual vs Predicted

```
plt.figure(figsize=(7, 7))

plt.scatter(
 y_valid,
 final_predictions,
 alpha=0.6
)

min_price = min(y_valid.min(), final_predictions.min())
max_price = max(y_valid.max(), final_predictions.max())

plt.plot(
```



```

 [min_price, max_price],
 [min_price, max_price],
 linestyle="--"
)

plt.xlabel("Actual SalePrice")
plt.ylabel("Predicted SalePrice")
plt.title("Actual vs Predicted House Prices")

plt.savefig(
 "../figures/actual_vs_predicted.png",
 dpi=150,
 bbox_inches="tight"
)

plt.show()

```

그리고 이미 있는 Feature Importance까지 하면 최소한:

```

figures/
├─ saleprice_distribution.png
├─ top_correlations.png
├─ actual_vs_predicted.png
└─ feature_importance.png

```

이렇게 4개가 돼.

나는 이 4개면 **Part 1** 제출용으로 충분히 좋아 보여. 너무 많은 그래프를 넣는 것보다:

- target은 어떻게 생겼나
- 어떤 feature가 관련 있나
- 모델이 실제값을 얼마나 잘 따라가나
- 모델은 어떤 feature를 중요하게 봤나

이 흐름이 딱 연결돼.

원하면 missing-value 그래프도 하나 추가할 수 있지만 **필수까진 아니야**. 지금은 속제 완성이 목표니까 일단 이 4개로 가자.

셋 다 다시 실행한 다음 터미널에서:

```
ls -lh figures
```

해봐. 4개 다 보이면 다음은 `results/` **확인** → **notebook clean-up** → **Run All**로 넘어가자.

2026. 9. 6. 오후 10:42:30



기존 셀들이 어떤것들일까?

2026. 9. 6. 오후 10:43:37



응. 우리가 앞에서 이미 만든 그래프 셀은 딱 이 4개였어. VS Code notebook에서 아래 코드들이 있는 셀을 찾아서 `plt.show()` 직전에 `plt.savefig(...)` 만 추가하면 돼.

## 1) SalePrice 분포 그래프

찾아야 하는 셀:

```
plt.figure(figsize=(8, 5))

plt.hist(df["SalePrice"], bins=30, edgecolor="black")

plt.xlabel("Sale Price")
plt.ylabel("Number of Houses")
plt.title("Distribution of SalePrice")

plt.show()
```

여기에:

```
plt.savefig(
 "../figures/saleprice_distribution.png",
 dpi=150,
 bbox_inches="tight"
)
```

를 `plt.show()` 바로 전에 추가.

## 2) Correlation Top Features 그래프

찾아야 하는 셀:

```

top_features = correlations.drop("SalePrice").head(10)

plt.figure(figsize=(10, 6))
plt.barh(top_features.index[::-1], top_features.values[::-1])
plt.xlabel("Correlation with SalePrice")
plt.title("Top Numerical Features Correlated with SalePrice")
plt.show()

```

`plt.show()` 전에:

```

plt.savefig(
 "../figures/top_correlations.png",
 dpi=150,
 bbox_inches="tight"
)

```

### 3) Actual vs Predicted 그래프

찾아야 하는 셀:

```

plt.figure(figsize=(7, 7))

plt.scatter(
 y_valid,
 final_predictions,
 alpha=0.6
)

min_price = min(y_valid.min(), final_predictions.min())
max_price = max(y_valid.max(), final_predictions.max())

plt.plot(
 [min_price, max_price],
 [min_price, max_price],
 linestyle="--"
)

```

```
plt.xlabel("Actual SalePrice")
plt.ylabel("Predicted SalePrice")
plt.title("Actual vs Predicted House Prices")

plt.show()
```

여기에:

```
plt.savefig(
 "../figures/actual_vs_predicted.png",
 dpi=150,
 bbox_inches="tight"
)
```

추가.

---

## 4) Feature Importance

이건 이미 저장했다고 했으니까 아마 이런 셀일 거야:

```
top_importance = feature_importance_df.head(15)

plt.figure(figsize=(10, 7))

plt.barh(
 top_importance["Feature"][:-1],
 top_importance["Importance"][:-1]
)

plt.xlabel("Feature Importance")
plt.title("Top 15 Gradient Boosting Feature Importances")

plt.savefig(
 "../figures/feature_importance.png",
 dpi=150,
 bbox_inches="tight"
)
```

```
plt.show()
```

## 가장 쉬운 찾는 방법

Notebook에서 `Cmd + F` 로 각각 검색하면 돼:

```
Distribution of SalePrice
Top Numerical Features
Actual vs Predicted
Feature Importances
```

그리고 우리가 missing value 시각화도 하나 만들었었어:

```
top_missing = missing_summary.head(15)

plt.figure(figsize=(10, 6))
plt.barh(
 top_missing.index[::-1],
 top_missing["MissingPercent"][::-1]
)

plt.xlabel("Missing Values (%)")
plt.ylabel("Feature")
plt.title("Top 15 Features with Missing Values")
plt.show()
```

이것도 꽤 괜찮은 그래프라 **5번째로 저장해도 좋아.**

```
plt.savefig(
 "../figures/missing_values.png",
 dpi=150,
 bbox_inches="tight"
)
```

그래서 최종적으로는:

figures/

|— saleprice\_distribution.png

|— missing\_values.png

|— top\_correlations.png

|— actual\_vs\_predicted.png

|— feature\_importance.png

이 5개면 흐름도 아주 깔끔해.

2026. 9. 6. 오후 10:43:42



좋아 끝

2026. 9. 6. 오후 10:45:23



좋아 ㅋㅋ 그럼 Part 1 분석은 거의 끝났어.

이제 남은 건 **정리와 제출 준비**야. 순서는:

1. `results/` 파일 확인
2. notebook에서 실패한 테스트 셀/중복 셀 정리
3. **Restart Kernel → Run All**
4. 결과 숫자 최종 확인
5. Part 1 `README.md` 작성
6. Git commit/push
7. chat transcript 저장
8. YouTube 찍기

먼저 터미널에서:

```
ls -lh results
```

그리고:

```
ls -lh figures
```

결과 보여줘.

그다음 내가 바로 **최종 notebook cleanup 체크리스트 + README 내용** 만들어줄게.

2026. 9. 6. 오후 10:45:27





```
(.venv) changhyunkim\@Mac house-prices % ls -lh results
total 80
-rw-r--r-- 1 changhyunkim staff 113B Sep 6 22:40
cross_validation_rmse.csv
-rw-r--r-- 1 changhyunkim staff 9.2K Sep 6 22:41
feature_importance.csv
-rw-r--r-- 1 changhyunkim staff 310B Sep 6 22:40
model_comparison.csv
-rw-r--r-- 1 changhyunkim staff 19K Sep 6 22:41
prediction_errors.csv
(.venv) changhyunkim\@Mac house-prices % ls -lh
figures
total 432
-rw-r--r-- 1 changhyunkim staff 70K Sep 6 22:44
actual_vs_predicted.png
-rw-r--r-- 1 changhyunkim staff 4.6K Sep 6 22:46
feature_importance.png
-rw-r--r-- 1 changhyunkim staff 56K Sep 6 22:45
missing_values.png
-rw-r--r-- 1 changhyunkim staff 26K Sep 6 22:44
saleprice_distribution.png
-rw-r--r-- 1 changhyunkim staff 47K Sep 6 22:44
top_correlations.png
```

2026. 9. 6. 오후 10:48:08



좋아. 파일 저장 상태는 정상이고 **Part 1 분석 artifact**도 충분히 갖춰졌어. 🍌

현재:

```
results/
├─ cross_validation_rmse.csv
├─ feature_importance.csv
├─ model_comparison.csv
└─ prediction_errors.csv
```

```
figures/
├─ actual_vs_predicted.png
├─ feature_importance.png
├─ missing_values.png
├─ saleprice_distribution.png
└─ top_correlations.png
```

이 정도면 아주 깔끔해.

이제 중요한 건 **Notebook** 최종 정리 → **Run All**이야.

## 1. Notebook에서 지울 것

우리가 중간에 테스트하면서 만들었던 것 중 최종본에는 굳이 필요 없는 셀들을 지워도 돼.

특히:

- `KeyError: 'PandasMissing'` 났던 실패 셀
- 같은 결과를 여러 번 출력한 중복 셀
- `print("Hello CMPE 255!")`
- `sys.executable` 확인용 셀
- package/kernel 테스트용 셀
- 완전히 동일한 그래프를 두 번 만든 셀

반면 `raw_df` 와 `clean_df` 를 만든 **missing-value investigation**은 남기자. 그게 이번 분석에서 꽤 좋은 부분이니까.

최종 Notebook 흐름은 대략 이렇게 보이면 된다:

1. Title / Project Introduction
2. Imports
3. Load Dataset
4. Data Understanding
  - shape

- data types
- duplicates
- missing values
- SalePrice summary

## 5. Target Analysis

- SalePrice histogram
- skewness

## 6. Missing Value Analysis

- missing-value %
- structural missing check
- raw NA / None investigation
- clean\_df creation

## 7. Exploratory Data Analysis

- correlation
- OverallQual vs SalePrice
- GrLivArea vs SalePrice

## 8. Data Preparation

- X / y
- train-validation split
- numerical / categorical features
- preprocessing pipelines

## 9. Modeling

- Dummy
- Ridge
- Random Forest
- Gradient Boosting

## 10. Model Comparison

## 11. Cross Validation

## 12. Prediction Error Analysis

## 13. Feature Importance

14. Final Conclusion

15. Save Results / Figures

이 흐름이면 교수님이 Notebook을 위에서 아래로 읽어도 이야기가 연결돼.

## 2. 그다음 진짜 중요한 작업

VS Code notebook 위쪽에서 **Restart → Run All** 해.

목적은:

*“지금까지 우연히 셀을 왔다 갔다 실행해서 돌아가는 게 아니라, 새 Python session에서 처음부터 끝까지 실행해도 돌아가는가?”*

를 확인하는 거야.

모든 셀에 초록 체크가 뜨고 에러가 없어야 해.

그리고 Run All 이후 최종 숫자가 우리가 지금 얻은 값과 같은지 확인해:

Gradient Boosting

RMSE  $\approx$  \$27,790

MAE  $\approx$  \$17,165

$R^2 \approx 0.899$

5-fold CV Mean RMSE

$\approx$  \$26,871

숫자가 같거나 사실상 동일하면 이 숫자를 공식 **Part 1 결과로 고정**하자.

## 3. Notebook 저장

Run All 성공 후:

Cmd + S

반드시 해줘.

그러면 `.ipynb` 안에 코드뿐 아니라 실행 결과와 그래프도 같이 저장돼. 교수님이 GitHub에서 notebook을 열면 바로 결과를 볼 수 있게 되는 거지.

---

그리고 그게 끝나면 더 이상 분석 코드는 건드리지 말자. 🔒

그 상태를 Final Analysis v1이라고 생각하면 돼.

그 다음 우리가 할 건:

**README 작성** → `requirements.txt` 생성 → 현재 **GitHub** 구조 확인 → **commit/push**

이거야.

일단 **Restart + Run All** 해봐. 에러 없이 끝나면 알려줘. 그러면 README를 이번에 나온 실제 결과로 바로 완성하자.

2026. 9. 6. 오후 10:48:16



```
results_df.to_csv(
 "../results/model_comparison.csv\",
 index=False
)
```

이런 친구들 몇개가 더 Conclusion 아래에 있는데 애네들 다 지울  
까?

코드 깃헙에 푸쉬하고 너가 검사해쥬 ㅎㅎ

2026. 9. 6. 오후 10:52:05



## Thinking

- 정리 방향 확인하기

저장 코드 셀은 삭제하지 않는 편이 좋아요. 실행 결과를 재현하려면 필요하니 `Save Final Artifacts` 같은  
별도 구역으로 옮기고, 결론은 마지막에 두면 구조가 깔끔해져요.

- 정리 방향 확인함

✓ 18s 동안 처리함

응 ㅎㅎ 그 `to_csv()` 셀들은 지우지 마. 오히려 최종 notebook에 남겨두는 게 좋아.

왜냐하면 저 코드들이:

```
results_df.to_csv(...)
feature_importance_df.to_csv(...)
error_df.to_csv(...)
```

처럼 **notebook**을 처음부터 **Run All** 했을 때 `results/` 파일까지 자동으로 다시 만들어주는 역할을 하  
거든. 재현성 측면에서 좋은 코드야.

다만 구조만 이렇게 정리하자:

```
...
Feature Importance
Prediction Error Analysis

Save Final Artifacts ← Markdown 제목 추가
[to_csv 코드들]
[필요한 savefig 코드들]

Final Conclusion ← 맨 마지막
Conclusion markdown
```

즉 **Conclusion** 밑에 코드가 주렁주렁 달리는 것보다는 **Conclusion**을 진짜 마지막으로 두는 게 훨씬 깔끔해.

그리고 이제 GitHub에 올리자. 다만 `.venv` 는 절대 **GitHub**에 올리면 안 돼.

현재 `house-prices` 폴더라면 repo root로:

```
cd ../../..
```

확인:

```
pwd
```

끝이:

```
FA2026_CMPE-255-Data-Mining
```

이어야 해.

먼저 `.venv` 가 ignore되는지 확인:

```
git check-ignore -q "Assignment 1/Part-1/house-prices/.venv" \
&& echo ".venv is ignored" \
|| echo ".venv is NOT ignored"
```

만약:

```
.venv is NOT ignored
```

가 나오면:

```
printf "\n# Local Python / Jupyter
files\n**/.venv/\n**/__pycache__/\n**/.ipynb_checkpoints/\n.DS_Store\n**/.DS_Store\n
>> .gitignore
```

그다음:

```
git status
```

그리고 새 Assignment 1만 stage:

```
git add .gitignore "Assignment 1"
```

무엇이 올라갈지 한번 확인:

```
git status
```

괜찮으면:

```
git commit -m "Rebuild Assignment 1 Part 1 house prices analysis"
```

마지막:

```
git push origin main
```

푸시 끝나면 그냥 “올렸어”라고 해줘.

그러면 내가 네 GitHub의 Assignment 1 에 직접 들어가서 특히:

- notebook이 GitHub에서 정상 표시되는지
- output이 실제로 저장되어 있는지
- 실패한 셀이 남아있는지
- results/ 4개 정상인지



- `figures/` 5개 정상인지
- `.venv` 가 실수로 올라가지 않았는지
- `intent/spec/plan` 흐름
- README와 실제 결과 숫자 일치 여부
- 교수님 요구사항 기준으로 빠진 것

까지 제출자 관점으로 한번 쪽 검사해줄게.

2026. 9. 6. 오후 10:52:16



올렸어

2026. 9. 6. 오후 10:54:26



## Thinking



웹 검색 중

- 확인 방법 정리

제공된 저장소 경로를 직접 확인하려 했지만 현재 페이지가 열리지 않아 과제 파일 내용을 검토하지 못하고 있어요. 저장소 링크나 핵심 파일 내용을 다시 보내주시면 요구사항과 구현 상태를 바로 비교해볼게요.

- 확인 방법 정리 완료



웹 검색 중

- 웹 검색 중

✓ 46s 동안 처리함

좋아 ㅎㅎ 이제 내가 **제출자 입장에서** 검사해보려고 GitHub를 열어봤는데, 지금 새 **Assignment 1** 경로가 공개 웹 쪽에서 아직 **cache miss**로 안 읽혀. 방금 push한 직후라 그런 것 같아. 내가 안 보이는데 본 척하진 않을게.

대신 네 로컬 repo는 이미 **origin/main**과 똑같은 상태니까, 터미널 결과로 **\*\*거의 똑같이 감사(audit)\*\***할 수 있어. repo root에서 아래 4개만 실행해서 그대로 붙여줘.

```
git status
```

```
git log -1 --oneline
```

```
git ls-files "Assignment 1" | sort
```

그리고 **.venv**가 실수로 올라갔는지:

```
git ls-files | grep -E '(^|/)\.venv(/|$)'
```

마지막 명령은 아무것도 안 나오는 게 정답이야.

추가로 notebook이 진짜 실행된 상태로 저장됐는지도 한 방에 확인하자. `house-prices` 폴더에서:

```
python - <<'PY'
import json
from pathlib import Path

p = next(Path("notebooks").glob("*.ipynb"))
nb = json.loads(p.read_text())

code_cells = [c for c in nb["cells"] if c["cell_type"] == "code"]
executed = [c for c in code_cells if c.get("execution_count") is not None]
with_outputs = [c for c in code_cells if c.get("outputs")]

print("Notebook:", p.name)
print("Code cells:", len(code_cells))
print("Executed cells:", len(executed))
print("Cells with outputs:", len(with_outputs))
PY
```

이것들 보여주면 내가 바로:

**폴더 구조 → notebook → results → figures → `.venv` → Git 상태 → 빠진 제출물**

순서로 체크해서  /  /  표 만들어줄게.

지금 단계에서 특히 보고 싶은 건 `README.md` 가 새 Part 1 결과인 **Gradient Boosting RMSE ≈ \$27,790 / CV ≈ \$26,871**로 아직 업데이트됐는지 여부야. README는 우리가 아직 제대로 손대지 않았으니까 아마 다음 작업 1순위일 가능성이 높아.

2026. 9. 6. 오후 10:54:31



```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
python - <<'PY'
import json
from pathlib import Path

p = next(Path("notebooks").glob("*.*ipynb"))
nb = json.loads(p.read_text())

code_cells = [c for c in nb["cells"] if c["cell_type"] ==
"code"]
executed = [c for c in code_cells if
c.get("execution_count") is not None]
with_outputs = [c for c in code_cells if c.get("outputs")]

print("Notebook:", p.name)
print("Code cells:", len(code_cells))
print("Executed cells:", len(executed))
print("Cells with outputs:", len(with_outputs))
PY
zsh: command not found: python
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
```

2026. 9. 6. 오후 10:56:10



응, 이건 간단해 ㅎㅎ

지금 repo root에서는 `.venv` 가 활성화되어 있지 않아서 `python` 명령이 없고, Mac에서는 보통 `python3` 를 써야 해.

그리고 내가 준 경로도 `notebooks/` 가 repo root 바로 아래에 있는 것처럼 써서 잘못됐어. 지금 실제 경로는:

```
Assignment 1/Part-1/house-prices/notebooks/
```

이니까 아래 명령으로 해줘.

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb"))
nb = json.loads(p.read_text())

code_cells = [c for c in nb["cells"] if c["cell_type"] == "code"]
executed = [c for c in code_cells if c.get("execution_count") is not None]
with_outputs = [c for c in code_cells if c.get("outputs")]

print("Notebook:", p.name)
print("Code cells:", len(code_cells))
print("Executed cells:", len(executed))
print("Cells with outputs:", len(with_outputs))
PY
```

정상이라면 대충 이런 식으로 나와:

```
Notebook: house_prices_analysis.ipynb
Code cells: XX
Executed cells: XX
Cells with outputs: XX
```

여기서 중요한 건:

```
Executed cells == Code cells
```

에 가깝게 나오는 거야.

그리고 아까 요청한 Git 상태도 같이 확인해줘:

```
git status
```

```
git log -1 --oneline
```

```
git ls-files "Assignment 1" | sort
```

마지막으로 `.venv` 가 올라갔는지:

```
git ls-files | grep -E '(^|/)\.venv(/|$)'
```

이건 **아무 출력이 없어야** 정상이야.

결과 붙여주면 내가 바로 최종 audit 해줄게.

2026. 9. 6. 오후 10:56:15



출력 없음 까지 확인했어

2026. 9. 6. 오후 10:57:00



## Thinking



웹 검색 중



웹 검색 중



몇 초간 처리함

좋아. 그럼 최소한 `.venv` 는 **GitHub**에 안 올라간 것 확인 완료야. 그건 잘 됐어.

다만 새 `Assignment 1` 경로는 웹에서 아직 직접 열리지 않아서, 나머지 검사는 네 로컬 Git 상태를 기준으로 보는 게 제일 정확해.

이제 딱 세 가지만 확인하면 돼:

```
git status
```

```
git log -1 --oneline
```

```
git ls-files "Assignment 1" | sort
```

그리고 notebook 실행 상태 확인 명령의 출력도 같이 보여줘:

```
Notebook: ...
```

```
Code cells: ...
```

```
Executed cells: ...
```

```
Cells with outputs: ...
```

이 네 결과만 붙여주면 내가 바로 **Part 1 제출 readiness**를  /  / 로 최종 검사해줄게.

지금까지 확인된 건:

- `.venv` 미추적 ✅
- results CSV 생성 ✅
- figures 5개 생성 ✅
- 모델링/교차검증/해석 완료 ✅
- 남은 핵심은 **notebook** 실행상태 + **Git**에 실제로 어떤 파일이 올라갔는지 + **README** 업데이트야.

2026. 9. 6. 오후 10:57:07





```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
git status
On branch main
Your branch is up to date with 'origin/main'.
```

Untracked files:

(use "git add \<file>..." to include in what will be committed)

&#x9;Assignment1/Part-1/

nothing added to commit but untracked files present (use "git add" to track)

```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
```

```
git log -1 --oneline
```

```
f777333 (**HEAD -> main**, **origin/main**,
origin/HEAD) Rebuild Assignment 1 Part 1 house
prices analysis
```

```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
```

```
git ls-files "Assignment 1" | sort
```

```
Assignment 1/Part-1/house-prices/chat-transcript/.gitkeep
```

```
Assignment 1/Part-1/house-prices/data/.gitkeep
```

```
Assignment 1/Part-1/house-prices/data/train.csv
```

```
Assignment 1/Part-1/house-prices/figures/.gitkeep
```

```
Assignment 1/Part-1/house-
prices/figures/actual_vs_predicted.png
```

```
Assignment 1/Part-1/house-
prices/figures/feature_importance.png
```

```
Assignment 1/Part-1/house-
prices/figures/missing_values.png
```

```
Assignment 1/Part-1/house-
prices/figures/saleprice_distribution.png
```

```
Assignment 1/Part-1/house-
prices/figures/top_correlations.png
```

```
Assignment 1/Part-1/house-prices/intent.md
```

```
Assignment 1/Part-1/house-prices/notebooks/.gitkeep
```

```
Assignment 1/Part-1/house-
```

```
prices/notebooks/house_prices_analysis.ipynb
Assignment 1/Part-1/house-prices/plan.md
Assignment 1/Part-1/house-prices/report/.gitkeep
Assignment 1/Part-1/house-prices/results/.gitkeep
Assignment 1/Part-1/house-prices/results/cross_validation_rmse.csv
Assignment 1/Part-1/house-prices/results/feature_importance.csv
Assignment 1/Part-1/house-prices/results/model_comparison.csv
Assignment 1/Part-1/house-prices/results/prediction_errors.csv
Assignment 1/Part-1/house-prices/spec.md
Assignment 1/Part-2/anomaly-detection/.gitkeep
Assignment 1/Part-2/customer-segmentation/.gitkeep
Assignment 1/Part-2/market-basket-analysis/.gitkeep
Assignment 1/README.md
```

2026. 9. 6. 오후 10:57:44



## Thinking



웹 검색 중



웹 검색 중



8s 동안 처리함

좋아, 이제 거의 다 확인됐어. 지금 상태는 새 `Assignment 1` 이 정상적으로 **commit/push** 된 상태야.

핵심은 이거야:

- `main` 브랜치가 `origin/main` 과 동일
- 마지막 commit이 `Rebuild Assignment 1 Part 1 house prices analysis`
- 새 `Assignment 1` 아래에 우리가 만든 Part 1 파일들이 전부 tracked
- `.venv` 는 tracked 안 됨



- figures 5개 ✅
- results 4개 ✅
- notebook tracked ✅
- Part 2 scaffold tracked ✅

즉 새 작업본은 잘 올라갔어.

다만 **한 가지 찝찝한 것**이 보여:

```
Untracked files:
 Assignment1/Part-1/
```

이건 우리가 백업으로 남겨두기로 한 **옛날 Assignment1** 폴더 안에 새로 생긴 **untracked** 파일들이 있다는 뜻이야.

새 제출본인:

```
Assignment 1/
```

에는 영향 없어.

하지만 `git status` 가 clean하지 않아서 나중에 실수로:

```
git add .
```

하면 옛날 백업 파일까지 같이 올라갈 수 있어.

그래서 지금부터는 **절대** `git add .` **하지 말고:**

```
git add "Assignment 1"
```

처럼 새 폴더만 지정하는 걸 추천해.

---

## 현재 **Part 1** 파일 구성

지금 Git이 실제로 추적하는 걸 보면 구조는 아주 좋아:

## Assignment 1/

- └─ README.md
- └─ Part-1/
  - └─ house-prices/
    - └─ intent.md
    - └─ spec.md
    - └─ plan.md
    - └─ data/
      - └─ train.csv
    - └─ notebooks/
      - └─ house\_prices\_analysis.ipynb
    - └─ figures/
      - └─ actual\_vs\_predicted.png
      - └─ feature\_importance.png
      - └─ missing\_values.png
      - └─ saleprice\_distribution.png
      - └─ top\_correlations.png
    - └─ results/
      - └─ cross\_validation\_rmse.csv
      - └─ feature\_importance.csv
      - └─ model\_comparison.csv
      - └─ prediction\_errors.csv
    - └─ chat-transcript/
    - └─ report/
  - └─ Part-2/
    - └─ customer-segmentation/
    - └─ market-basket-analysis/
    - └─ anomaly-detection/

이건 교수님이 말한:

*"Make sure you have all artifacts captured in a nice README.md file in your github directory"*

를 만족시키기 좋은 형태야.

## 지금 남은 체크 1: Notebook 실행 상태

이거 하나만 아직 확인을 못 했어.

repo root에서 아래 명령 다시 실행해줘:

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

code_cells = [
 c for c in nb["cells"]
 if c["cell_type"] == "code"
]

executed = [
 c for c in code_cells
 if c.get("execution_count") is not None
]

with_outputs = [
 c for c in code_cells
 if c.get("outputs")
]

print("Notebook:", p.name)
print("Code cells:", len(code_cells))
```

```
print("Executed cells:", len(executed))
print("Cells with outputs:", len(with_outputs))
PY
```

여기서 예를 들어:

```
Code cells: 26
Executed cells: 26
```

처럼 나오면 좋고.

`with_outputs` 는 모든 코드셀이 output을 생성하는 건 아니니까 code cells보다 적어도 정상.

## 그리고 사실 지금 제일 먼저 해야 할 건 README야

현재 Assignment 1/README.md 는 tracked되어 있지만, 우리가 이번 새 분석 결과를 기준으로 다시 작성한 건 아직 아니잖아.

그래서 Part 1 분석을 더 건드리는 것보다 이제 README를 업데이트하는 게 맞아.

이번 새 실행에서 확정된 숫자는:

Model	RMSE	대략적인 결과
Dummy	~\$87,619	baseline
Ridge	~\$29,721	크게 개선
Random Forest	~\$29,024	Ridge보다 조금 개선
<b>Gradient Boosting</b>	<b>~\$27,790</b>	<b>Best validation model</b>

그리고:

```
Gradient Boosting MAE ≈ $17,165
Gradient Boosting R² ≈ 0.899
```

```
5-fold CV mean RMSE ≈ $26,871
CV std ≈ $3,651
```

Feature importance도:

OverallQual  $\approx$  50%

GrLivArea  $\approx$  15%

가 핵심.

이제 이걸 README에 넣으면 돼.

## 현재 내 Part 1 판정

항목	상태
Project organization	✓
Intent/spec/plan	✓
Kaggle dataset	✓
Data Understanding	✓
EDA	✓
Missing-value investigation	✓
Preprocessing	✓
Multiple models	✓
Baseline comparison	✓
Validation metrics	✓
Cross-validation	✓
Error analysis	✓
Feature interpretation	✓
Figures	✓
Result CSVs	✓

항목	상태
Executed notebook	🔍 마지막 확인
README	⚠️ 업데이트 필요
AI transcript	❌ 아직
YouTube	❌ 마지막
YouTube link in README	❌ 마지막

그러니까 **Data Science** 작업 자체는 거의 끝났어.

이제부터는 분석 추가보다는 **제출 패키징** 단계야.

Notebook 실행 count만 확인해주면, 다음 답변에서 내가 바로 `Assignment 1/README.md` **최종본을 통째로 작성해줄게**. 그걸 넣고 GitHub push한 다음 Part 2로 넘어가자.

2026. 9. 6. 오후 10:57:54





```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-
prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

code_cells = [
 c for c in nb["cells"]
 if c["cell_type"] == "code"
]

executed = [
 c for c in code_cells
 if c.get("execution_count") is not None
]

with_outputs = [
 c for c in code_cells
 if c.get("outputs")
]

print("Notebook:", p.name)
print("Code cells:", len(code_cells))
print("Executed cells:", len(executed))
print("Cells with outputs:", len(with_outputs))
PY
Notebook: house_prices_analysis.ipynb
Code cells: 53
Executed cells: 52
Cells with outputs: 36
```



좋아, 거의 완벽해. **53개 코드 셀 중 52개 실행**, output 있는 셀은 36개면 정상 범위야.

지금 딱 하나만 확인하면 돼:

### 실행 안 된 코드 셀 1개가 무엇인지 찾기

repo root에서 이거 실행해줘:

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

for i, cell in enumerate(nb["cells"]):
 if cell["cell_type"] == "code" and cell.get("execution_count") is None:
 print(f"Unexecuted code cell index: {i}")
 print("Source:")
 print("".join(cell.get("source", [])))
 print("-" * 60)

PY
```

그러면 아마 코드 하나가 딱 뜰 거야.

그게 예를 들어:

```
some comment
```

처럼 사실상 빈 셀이면 **삭제해도 되고**,

실제 코드가 있다면 notebook에서 그 셀 실행한 뒤:

Restart Kernel

→ Run All

→ Save

한 번 더 하면 돼.

그리고 **Cells with outputs: 36** 은 전혀 문제없어. 예를 들어:

```
results_df.to_csv(...)
```

같은 코드는 실행돼도 화면 출력이 없으니까 output count가 code cell count보다 적은 게 정상이지.

최종 목표는:

Code cells: 53

Executed cells: 53

또는 빈/불필요 셀 하나를 삭제해서:

Code cells: 52

Executed cells: 52

가 되면 끝.

그거만 잡고 나면 **Part 1 notebook은 freeze**하자. 더 이상 모델이나 숫자 안 건드리고, 바로

**README.md** 최종본 작성으로 넘어가면 돼.

2026. 9. 6. 오후 10:58:51



```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-
prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

for i, cell in enumerate(nb["cells"]):
 if cell["cell_type"] == "code" and
cell.get("execution_count") is None:
 print(f"Unexecuted code cell index: {i}")
 print("Source:")
 print("".join(cell.get("source", [])))
 print("-" * 60)

PY
Unexecuted code cell index: 63
Source:
```

2026. 9. 6. 오후 10:59:20



좋아, 이건 **완전 빈 코드 셀 하나**야.

출력이:

```
Unexecuted code cell index: 63
Source:
```

이렇게 비어 있으니까, notebook 맨 아래쪽 어딘가에 생성된 **빈 code cell**이야.

그냥 그 셀 삭제하면 돼. VS Code notebook에서 맨 아래로 가서 빈 코드 셀 하나 찾아서 휴지통 아이콘으로 지우거나, 셀 선택 후 삭제해.

그다음:

```
Restart Kernel
```

```
→ Run All
```

```
→ Save
```

한 번 더 하고, 다시 확인하면 아마:

```
Code cells: 52
```

```
Executed cells: 52
```

로 맞을 거야.

이제 진짜 notebook은 끝났다고 봐도 돼 ㅋㅋ.

다음은 바로 **README** 최종본 작성 가자.

*2026. 9. 6. 오후 10:59:27*



```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-
prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

for i, cell in enumerate(nb["cells"]):
 if cell["cell_type"] == "code" and
cell.get("execution_count") is None:
 print(f"Unexecuted code cell index: {i}")
 print("Source:")
 print("".join(cell.get("source", [])))
 print("-" * 60)
PY
Unexecuted code cell index: 63
Source:

이거 실행하고 아무것도 안뜨면 된거지?
```

2026. 9. 6. 오후 11:00:40



맞아. 이 명령을 실행했을 때 아무것도 출력되지 않으면 완료야.

지금 내가 붙여준 결과는 아직:

```
Unexecuted code cell index: 63
Source:
```

가 뜨니까 빈 코드 셀 하나가 아직 남아 있는 상태야.

Notebook에서 그 빈 셀을 삭제한 다음:

```
Restart Kernel → Run All → Save
```

하고 같은 검사 명령을 다시 실행해.

최종적으로 터미널에 그냥 프롬프트만 돌아오면:

```
changhyunkim@Mac FA2026_CMPE-255-Data-Mining %
```

✅ 모든 코드 셀이 실행된 상태라고 보면 돼.

그렇게 되면 notebook은 이제 freeze하고 README로 넘어가자.

2026. 9. 6. 오후 11:00:51



노트북에서 빈 셀 못찾겠는데!!? 좋은 방법이 없을까?

2026. 9. 6. 오후 11:02:07



응 ㅋㅋ 그럼 **노트북 UI**에서 찾지 말고 **파일**에서 빈 코드 셀만 자동으로 지워버리자. 이게 제일 빠르고 안전해.

repo root에서 아래 실행해:

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

before = len(nb["cells"])

nb["cells"] = [
 cell for cell in nb["cells"]
 if not (
 cell["cell_type"] == "code"
 and not "".join(cell.get("source", [])).strip()
)
]

after = len(nb["cells"])

p.write_text(json.dumps(nb, indent=1))

print("Notebook:", p.name)
print("Cells before:", before)
```



```
print("Cells after:", after)
print("Removed:", before - after)
PY
```

이 코드는 내용이 완전히 비어 있는 **code cell**만 삭제해. 정상 코드 셀은 안 건드려.

예를 들어:

```
Cells before: 64
Cells after: 63
Removed: 1
```

이렇게 나오면 성공.

그 다음 VS Code에서 notebook을 다시 열거나 reload하고:

**Restart Kernel → Run All → Save**

한 번 해줘.

그리고 다시 검사:

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

for i, cell in enumerate(nb["cells"]):
 if cell["cell_type"] == "code" and cell.get("execution_count") is None:
 print(f"Unexecuted code cell index: {i}")
 print("Source:")
 print("".join(cell.get("source", [])))
PY
```

아무것도 안 뜨면 끝.

이게 되면 이제 notebook은 진짜 손 떼고 README로 넘어가자 ㅋㅋ.

2026. 9. 6. 오후 11:02:13



PY

Notebook: house\\_prices\\_analysis.ipynb

Cells before: 67

Cells after: 67

Removed: 0

로 나오는데!?

2026. 9. 6. 오후 11:02:54



ㅋㅋ 이걸 좀 이상하네. 겉으로는 빈 셀처럼 보이는데 실제 `source` 안에 우리가 못 본 특수문자나 다른 형태가 들어있는 것 같아.

지우기 전에 정확히 정체를부터 보자. repo root에서 이걸 실행해줘:

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

for i, cell in enumerate(nb["cells"]):
 if cell["cell_type"] == "code" and cell.get("execution_count") is None:
 print("Cell index:", i)
 print("Source repr:", repr(cell.get("source")))
 print("Outputs:", repr(cell.get("outputs")))
 print("Metadata:", repr(cell.get("metadata")))
 print("Keys:", cell.keys())
 print("-" * 60)
```

PY

`repr()` 을 쓰면 눈에 안 보이는 것도 보여줘.

예를 들어:

```
Source repr: ['\n']
```

또는:

```
Source repr: [' ']
```

혹은 뭔가 다른 형태가 나올 수 있어.

지금 `Removed: 0` 이 나온 걸 보면 우리가 생각한 “완전 빈 **source**”와 실제 **cell** 상태가 약간 다른 것 같아.

결과 보여줘. 그럼 그 셀만 **index 63** 기준으로 정확하게 삭제하는 코드 줄게. 다른 셀은 절대 안 건드리게.

2026. 9. 6. 오후 11:03:01



```
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-
prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

for i, cell in enumerate(nb["cells"]):
 if cell["cell_type"] == "code" and
cell.get("execution_count") is None:
 print("Cell index:", i)
 print("Source repr:", repr(cell.get("source")))
 print("Outputs:", repr(cell.get("outputs")))
 print("Metadata:", repr(cell.get("metadata")))
 print("Keys:", cell.keys())
 print("-" * 60)

PY
changhyunkim\@Mac FA2026_CMPE-255-Data-Mining %
```

2026. 9. 6. 오후 11:03:27



응, 이번엔 진짜 끝난 거야. ㅋㅋ

방금 명령은:

`execution_count is None` 인 코드 셀을 전부 찾아라

였는데 아무것도 출력되지 않았으니까, 현재 **notebook**에는 실행 안 된 코드 셀이 **0개**라는 뜻이야.

아까 index 63 이 뜬 건 그 시점의 notebook 상태였고, 이후에 VS Code에서 Run All / Save 하면서 상태가 갱신된 것 같아.

마지막으로 숫자만 한번 확인하자:

```
python3 - <<'PY'
import json
from pathlib import Path

p = next(
 Path("Assignment 1/Part-1/house-prices/notebooks").glob("*.ipynb")
)

nb = json.loads(p.read_text())

code_cells = [
 c for c in nb["cells"]
 if c["cell_type"] == "code"
]

executed = [
 c for c in code_cells
 if c.get("execution_count") is not None
]

print("Code cells:", len(code_cells))
print("Executed cells:", len(executed))
PY
```

예를 들어:

```
Code cells: 53
Executed cells: 53
```

처럼 같으면 **Notebook 검증 완료** ✅

그리고 한 가지 중요해. 우리가 push한 뒤 notebook을 다시 Save/Run All 했으니까 **GitHub에는 최신 notebook 변경사항이 아직 안 올라갔을 수 있어.**

바로:

```
git status
```

해봐.

Assignment 1/.../house\_prices\_analysis.ipynb 가 modified로 뜨면:

```
git add "Assignment 1"
git commit -m "Finalize executed Part 1 notebook"
git push origin main
```

해주면 돼.

그 다음에는 진짜 **README** 작성 → **Part 1** 마감으로 넘어가자.

2026. 9. 6. 오후 11:03:35



오키 끝. 리드미 가자

2026. 9. 6. 오후 11:04:15



좋아 ㅋㅋ 이제 분석은 **freeze**하고 제출용 **README** 만들자.

지금은 Part 1은 완성 상태고 Part 2는 앞으로 할 거니까, README도 **Part 1**은 실제 결과를 쓰고 **Part 2**는 진행 예정으로만 표시하는 게 정직하고 깔끔해.

repo root에서 아래처럼 열어:

```
nano "Assignment 1/README.md"
```

기존 내용 지우고 아래를 통째로 넣자.

```
CMPE 255 – Assignment 1
```

```
AI-Assisted Data Science
```

```
This repository contains my work for CMPE 255 Assignment 1.
```

```
The assignment explores how an agentic AI coding assistant can support an end-to-end data science workflow while the student remains responsible for executing, validating, understanding, and explaining the results.
```

```
The project is organized into two parts:
```

- **Part 1:** AI-assisted end-to-end data science using a Kaggle dataset
- **Part 2:** Replication of selected data science experiments from the professor's `data\_science\_examples` repository

```

```

```
Part 1 – House Price Prediction
```

```
Project Overview
```



For Part 1, I used the Kaggle **House Prices: Advanced Regression Techniques** dataset to build a supervised machine learning workflow for predicting residential sale prices.

The target variable is:

``SalePrice``

The analysis follows a workflow inspired by CRISP-DM:

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling
5. Evaluation
6. Interpretation and Communication

AI tools were used to assist with planning, code generation, debugging, analysis, and documentation. However, generated code and results were executed, checked, and interpreted before being included in the final project.

---

## Dataset

The training dataset contains:

- **1,460 observations**
- **81 columns**
- Numerical and categorical features
- **0 duplicate rows**

The target variable ``SalePrice`` is strongly right-skewed, with a skewness of approximately **1.88**.

An important part of the data-cleaning process was investigating missing values. Several values that initially appeared to be missing actually represented the absence of property features.

Examples include:

- `PoolQC = NA` → no pool
- `GarageType = NA` → no garage
- `BsmtQual = NA` → no basement
- `MasVnrType = None` → no masonry veneer

This investigation helped avoid incorrectly treating meaningful property information as generic missing data.

---

## ## Exploratory Data Analysis

The numerical features most strongly correlated with `SalePrice` included:

- `OverallQual` – approximately **0.79**
- `GrLivArea` – approximately **0.71**
- `GarageCars` – approximately **0.64**
- `GarageArea` – approximately **0.62**
- `TotalBsmtSF` – approximately **0.61**

The analysis suggests that overall property quality and usable living space are strongly associated with house sale prices.

---

## ## Data Preparation

The preprocessing workflow handled numerical and categorical variables separately.

Numerical features were processed using:

- Missing-value imputation
- Feature scaling where appropriate

Categorical features were processed using:

- Domain-aware missing-value handling
- Missing-value imputation where necessary
- One-hot encoding

Scikit-learn `Pipeline` and `ColumnTransformer` were used to make the preprocessing and modeling workflow reproducible.

---

## ## Models Evaluated

Four regression approaches were evaluated:

1. Dummy Regressor
2. Ridge Regression
3. Random Forest Regressor
4. Gradient Boosting Regressor

The Dummy Regressor was used as a baseline so that the machine learning models could be compared against a simple average-price prediction strategy.

The complete model comparison is available here:

[[`results/model\\_comparison.csv`](#)]([Part-1/house-prices/results/model\\_comparison.csv](#))

---

## ## Best Model – Gradient Boosting

Gradient Boosting achieved the strongest validation performance.

Approximate validation results:

- **RMSE:** \$27,790
- **MAE:** \$17,165
- **R<sup>2</sup>:** 0.899

The model substantially outperformed the Dummy baseline, which produced an RMSE of approximately \$87,619.

---

## ## Cross-Validation

To determine whether the Gradient Boosting performance depended heavily on one train-validation split, I also performed 5-fold cross-validation.

Results:

- **Mean CV RMSE:** approximately \$26,871
- **Standard deviation:** approximately \$3,651

The cross-validation result was similar to the original validation result, providing additional evidence that the model generalizes reasonably well across different subsets of the dataset.

Detailed results:

```
[`results/cross_validation_rmse.csv`](Part-1/house-prices/results/cross_validation_rmse.csv)
```

---

## ## Model Interpretation

The Gradient Boosting feature-importance analysis identified the following as some of the strongest predictive features:

- ``OverallQual`` – approximately 50% of total feature importance
- ``GrLivArea`` – approximately 15%
- ``GarageCars``
- ``TotalBsmtSF``
- ``BsmtFinSF1``
- ``1stFlrSF``

These findings are consistent with the earlier correlation analysis.

Feature importance describes how the model uses variables for prediction and should not be interpreted as evidence of causation.

---

## ## Prediction Error Analysis

Although the final model performed well overall, some individual predictions contained large errors.

Several of the largest underpredictions occurred for expensive homes. For example, one property sold for approximately \$611,657 but was predicted at approximately \$403,266.

This suggests that unusual or high-priced properties may contain characteristics that are more difficult for the model to capture.

The complete prediction-error results are available here:

```
[`results/prediction_errors.csv`](Part-1/house-prices/results/prediction_errors.csv)
```

---

## ## Visualizations

### ### SalePrice Distribution

```
![SalePrice Distribution](Part-1/house-prices/figures/saleprice_distribution.png)
```

### ### Missing Values

```
![Missing Values](Part-1/house-prices/figures/missing_values.png)
```

### ### Numerical Correlations

```
![Top Correlations](Part-1/house-prices/figures/top_correlations.png)
```

### ### Actual vs Predicted Prices

```
![Actual vs Predicted](Part-1/house-prices/figures/actual_vs_predicted.png)
```

### ### Gradient Boosting Feature Importance

```
![Feature Importance](Part-1/house-prices/figures/feature_importance.png)
```

---

## ## Part 1 Project Artifacts

### ### Notebook

```
[`house_prices_analysis.ipynb`](Part-1/house-prices/notebooks/house_prices_analysis.ipynb)
```

The final notebook was restarted and executed from beginning to end to verify that the analysis is reproducible.

### ### Dataset

```
[`train.csv`](Part-1/house-prices/data/train.csv)
```

### ### Results

- [`model\_comparison.csv`](Part-1/house-prices/results/model\_comparison.csv)
- [`cross\_validation\_rmse.csv`](Part-1/house-prices/results/cross\_validation\_rmse.csv)
- [`prediction\_errors.csv`](Part-1/house-prices/results/prediction\_errors.csv)
- [`feature\_importance.csv`](Part-1/house-prices/results/feature\_importance.csv)

### ### Project Planning

- [`intent.md`](Part-1/house-prices/intent.md)
- [`spec.md`](Part-1/house-prices/spec.md)
- [`plan.md`](Part-1/house-prices/plan.md)

These planning documents are supplemental artifacts used to organize the

AI-assisted workflow.

---

## ## AI-Assisted Workflow

The AI assistant was used for tasks such as:

- Planning the data science workflow
- Suggesting exploratory analysis
- Generating Python code
- Debugging notebook issues
- Designing preprocessing pipelines
- Comparing regression models
- Interpreting model results
- Organizing documentation

AI output was not accepted blindly.

For example, during missing-value analysis, additional inspection revealed that Pandas' default parsing behavior could interpret meaningful values such as `NA` and `None` as missing data. The preprocessing approach was adjusted after examining the actual domain meaning of those values.

The final explanations in this project are based on my interpretation of the executed results.

---

## # Part 2 – Data Science Experiment Replication

Part 2 will replicate selected experiments from the professor's `data\_science\_examples` repository using an AI coding assistant.

The planned experiments are:

### ### Customer Segmentation

Location:

`Part-2/customer-segmentation/`

### Market Basket Analysis

Location:

`Part-2/market-basket-analysis/`

### Anomaly Detection

Location:

`Part-2/anomaly-detection/`

Part 2 results and artifacts will be added after each experiment is executed and validated.

---

# YouTube Walkthrough

The final Assignment 1 walkthrough will explain the end-to-end AI-assisted data science process and the results in my own words.

**\*\*YouTube Link:\*\*** To be added after recording.

---

# Assignment Status

## Part 1

**\*\*Analysis complete\*\***

The House Prices notebook, figures, results, and supporting project artifacts are included in this repository.

## Part 2



**\*\*In progress\*\***

The experiment replication artifacts will be added before final submission.

---

## Author

Changhyun Kim  
San José State University  
CMPE 255 — Data Mining  
Fall 2026

저장:

```
Control + O
Enter
Control + X
```

그다음 GitHub에서 이미지까지 제대로 뜨는지 확인하기 전에 로컬에서:

```
git diff -- "Assignment 1/README.md"
```

한번 보고 괜찮으면:

```
git add "Assignment 1/README.md"
git commit -m "Update Assignment 1 README with Part 1 results"
git push origin main
```

한 가지 다음에 추가해야 할 것

현재 Part 1에서 교수님 요구사항 중 아직 비어 있는 건 **AI chat transcript**야.

지금:

chat-transcript/

└─ .gitkeep

만 있으니까, 최종 제출 전에는 이 대화 기록을 PDF든 document든 넣어야 해.

하지만 지금은 **Part 1 코드/README**까지 완료하고 바로 **Part 2**로 넘어가자.

YouTube랑 transcript는 모든 분석 끝내고 마지막에 한 번에 패키징하는 게 훨씬 효율적이야.

2026. 9. 6. 오후 11:04:34



## Thinking

- 재구성 방향 정리

현재 대화에서 확인되는 내용만 바탕으로 영어 요약과 주요 상호작용을 재구성하려 하고 있어요. 전체 기록이 보이지 않으므로, 확인 가능한 범위 안에서만 정확하게 정리하겠습니다.

- 재구성 방향을 정리했습니다

✓ 14s 동안 처리함

2026. 9. 6. 오후 11:37:04